

Centro educativo

Código	Centro	Concello	Ano académico
15005397	I.E.S. Fernando Wirtz Suárez	A Coruña	2025-2026

Ciclo formativo

Código da familia profesional	Familia profesional	Código do ciclo formativo	Ciclo formativo	Grao	Réxime
FP16	Informática e Comunicacóns	SIFC03	Desenvolvemento de Aplicacións Web	Superior	Dual

Módulo profesional

Código MP/UF	Nome
MP0616	Proxecto de Desenvolvemento de Aplicación Web Equivalencia en créditos ECTS: 5. Código: MP0616. Duración: 26 horas

Profesorado responsable

Titor	Prado Espiñeira, Fernando
Equipo docente	García Ulla, Constantino Gimeno Peón, María Jesús Iglesias Gómez, Marta Pascual Vázquez, Víctor Rodríguez Diéguez, Fernando

Alumna

Alumna	Sara Barreiro Riveiro
---------------	-----------------------

Datos do proxecto

Título	Explora
---------------	---------

Índice

Índice	2
1. Obxectivo	4
1.1. Contexto e motivación	4
1.2. Obxectivos xerais e específicos	5
2. Descrición	7
2.1. Descrición xeral do problema	7
2.2. Requisitos funcionais	8
2.3. Requisitos non funcionais	10
2.3. Análise de usuaria	12
2.4. Casos de uso	15
3. Alcance	20
3.1. Funcionalidades incluídas	20
3.2. Aspectos técnicos incluídos no proxecto	22
3.3. Limitacións do alcance	22
4. Planificación	25
4.1. Metodoloxía de desenvolvemento	25
4.2. Organización en sprints	26
4.3. Planificación detallada dos sprints	26
4.4. Fitos do proxecto	33
4.5. Diagrama de Gantt	35
5. Medios a utilizar	36

5.1. Tecnoloxías	36
5.2. Ferramentas	38
6. Presuposto	39
6.1. Custos de infraestrutura	39
6.2. Custos de ferramentas e servizos	40
6.3. Custo de desenvolvemento	40
6.4. Resumo de custos totais	41
7. Execución	42
7.1. Deseño do sistema	42
7.2. Implementación	46
7.3. Probas e validación	48
7.4. Despregamento e documentación	53
7.5. Resultados e avaliación	70
7.6. Conclusións e liñas de mellora	73
Anexos	76
1. Diagrama da arquitectura do backend	76
2. Diagrama da arquitectura do frontend	76
3. Modelo Entidade-Relación Conceptual de Chen	77
4. Diagrama UML de clases	77
5. Mockups	78
6. Guía de estilo	80

1. Obxectivo

1.1. Contexto e motivación

Filosofía: privacidade, liberdade e independencia tecnolóxica

Na actualidade, as solucións web máis usadas para mapas dixitais implican unha gran dependencia de servizos centralizados e a cesión masiva de datos persoais a grandes empresas tecnolóxicas. Esta situación non só limita a soberanía das persoas usuarias sobre os seus propios datos, senón que tamén expón a información sensible a riscos de privacidade e uso indebido.

Por iso, existe a necesidade de crear alternativas baseadas en software libre, colaborativas e respectuosas coa privacidade, que lle permitan ás persoas usuarias controlar de forma íntegra os seus datos e contidos xeoespaciais. Neste sentido, OpenStreetMap constitúe un exemplo paradigmático dunha plataforma aberta e comunitaria que posibilita este tipo de desenvolvemento.

Este proxecto pretende ofrecer unha aplicación web de mapas colaborativos construída sobre tecnoloxías libres, cunha filosofía clara de control total da usuaria sobre a súa información persoal e os seus contidos. A aplicación adopta un enfoque de *privacy by design*, minimizando a recollida de datos persoais e aplicando mecanismos de seguridade que garantan a confidencialidade da información das usuarias.

Baleiro no mercado: a colaboración

Aínda que existen ferramentas que permiten compartir mapas ou localizacións, moitas delas presentan limitacións en aspectos como o control granular de permisos, a edición colaborativa estruturada ou a independencia respecto de plataformas propietarias.

Este proxecto busca cubrir esa fenda mediante o desenvolvemento dunha aplicación que permite crear e xestionar mapas personalizados, públicos ou privados, con múltiples usuarias colaborando segundo un sistema xerárquico de roles e permisos. O obxectivo é facilitar o traballo en equipo sobre información xeográfica de forma estruturada, sen depender de servizos centralizados alleos ao control das persoas usuarias.

1.2. Obxectivos xerais e específicos

Obxectivos xerais

Desenvolver unha aplicación web de mapas colaborativos baseada en tecnoloxías libres que lles permita ás usuarias crear, xestionar e compartir mapas públicos ou privados, garantindo a privacidade da información e facilitando a colaboración entre múltiples usuarias mediante un sistema estruturado de permisos e funcionalidades accesibles.

Obxectivos específicos

Establécense os seguintes obxectivos específicos, que describen as funcionalidades e características principais que debe incorporar a aplicación desenvolvida:

1. Integrar mapas base de OpenStreetMap como referencia para a visualización e edición dos mapas colaborativos dentro da aplicación.
2. Permitir a creación, configuración e xestión de mapas personalizados, con opción de definir a súa visibilidade como pública ou privada.
3. Desenvolver un sistema de descubrimento de mapas públicos baseado na localización xeográfica, que lles permita ás persoas usuarias buscar mapas asociados a unha cidade, rexión ou lugar concreto e visualizar os mapas colaborativos que teñan esa localización como referencia.
4. Implementar un sistema de convite e colaboración entre usuarias, permitindo asignar diferentes roles dentro de cada mapa.
5. Desenvolver ferramentas colaborativas que permitan engadir, modificar e eliminar marcadores xeográficos, mantendo un historial de cambios que posibilite a consulta e restauración de versións anteriores.
6. Integrar a posibilidade de engadir notas asociadas aos marcadores, permitindo definir a súa visibilidade segundo o contexto do mapa ou da persoa usuaria.
7. Implementar un sistema de categorías para os marcadores, con atributos visuais como cor e iconas, que facilite a organización da información e o filtrado personalizado dos elementos do mapa.
8. Diseñar e implementar un sistema de xestión de usuarias e control de acceso, baseado en autenticación e autorización mediante roles que regulen os permisos de cada participante.

9. Desenvolver un perfil de usuaria que permita xestionar información persoal e preferencias da conta.
10. Implementar funcionalidade *offline*, permitindo a descarga previa de información para a consulta e edición de datos mesmo en ausencia de conexión a internet.
11. Garantir a seguridade e privacidade da información mediante mecanismos de autenticación, control de acceso e cifrado das comunicacións.

2. Descrición

2.1. Descrición xeral do problema

Na actualidade existen numerosas plataformas que permiten visualizar e interactuar con mapas dixitais a través da web. Estas ferramentas utilízanse habitualmente para localizar lugares, planificar desprazamentos ou consultar información xeográfica. Con todo, moitas destas solucións céntranse principalmente na navegación individual e na consulta de información, e non proporcionan mecanismos completos para a creación e xestión de mapas personalizados de forma colaborativa entre múltiples usuarias.

Ademais, unha gran parte das plataformas máis utilizadas dependen de servizos centralizados pertencentes a grandes empresas tecnolóxicas, o que implica a utilización de APIs privativas e a cesión de determinados datos de uso por parte das usuarias. Esta dependencia tecnolóxica limita a capacidade de control sobre a información xerada e dificulta a creación de solucións abertas que poidan ser adaptadas ou ampliadas segundo necesidades específicas.

Por outra banda, as ferramentas existentes adoitan presentar limitacións no ámbito da colaboración estruturada. Aínda que algúns servizos permiten compartir mapas ou localizacións, non adoitan ofrecer sistemas avanzados de permisos, edición colaborativa organizada ou historial detallado de cambios, funcionalidades que resultan necesarias cando varias persoas participan na construción e mantemento dun mesmo mapa.

Ante esta situación, xorde a necesidade de desenvolver unha aplicación que permita crear e xestionar mapas colaborativos de forma flexible, controlada e baseada en tecnoloxías abertas, proporcionándolles ás usuarias ferramentas para engadir marcadores, organizar información mediante categorías e colaborar con outros participantes segundo distintos niveis de permisos. O proxecto presentado aborda este problema mediante o desenvolvemento dunha aplicación web que integra mapas base de OpenStreetMap, incorpora funcionalidades colaborativas e garante o control das usuarias sobre a información xeoespacial que crean e comparten.

2.2. Requisitos funcionais

Xestión de usuarias

RF-01. Rexistro de usuarias. O sistema debe permitir o rexistro de novas usuarias mediante a introdución de información básica de identificación.

RF-02. Autenticación de usuarias. O sistema debe permitir que as usuarias rexistradas inicien sesión para acceder ás funcionalidades da aplicación.

RF-03. Activación da conta mediante correo electrónico. O sistema debe garantir que tras o rexistro se envía un token ao correo electrónico como requisito para o inicio de sesión.

RF-04. Xestión do perfil de usuaria. O sistema debe permitirlles ás persoas usuarias modificar a información do seu perfil, incluíndo nome, correo electrónico, nome de usuaria, contrasinal e preferencia de idioma da interface.

RF-05. Preferencias de idioma. O sistema debe permitir gardar e modificar a preferencia de idioma da interface, persistíndoa localmente e sincronizándoa co perfil da usuaria no servidor.

Xestión de mapas colaborativos

RF-06. Creación de mapas. O sistema debe permitirlles ás usuarias rexistradas crear novos mapas especificando nome, descrición, configuración de visibilidade e localización.

RF-07. Consulta de mapas. O sistema debe permitirlles ás persoas usuarias consultar a lista de mapas aos que teñen acceso, organizados en tres categorías diferenciadas: mapas creados pola usuaria, mapas nos que colabora e mapas gardados.

RF-08. Edición de mapas. O sistema debe permitir modificar a información e configuración dos mapas creados pola usuaria.

RF-09. Eliminación de mapas. O sistema debe permitir eliminar mapas creados pola usuaria.

RF-10. Configuración de visibilidade dos mapas. O sistema debe permitir definir se un mapa é público ou privado.

RF-11. Visualización cartográfica. O sistema debe permitir visualizar mapas base e elementos xeográficos mediante unha interface cartográfica interactiva.

RF-12. Convite a usuarias. O sistema debe permitir convidar a outras usuarias a participar nun mapa público ou privado mediante un mecanismo de invitación.

RF-13. Xestión de roles por mapa. O sistema debe permitir asignarlles diferentes roles ás usuarias dentro dun mapa colaborativo.

RF-14. Control de acceso aos mapas. O sistema debe restrinxir a visualización e modificación dos mapas en función dos permisos asignados a cada usuaria.

RF-15. Gardado de mapas. O sistema debe permitirlle á usuaria gardar mapas públicos, co obxectivo de facilitar a consulta dos seus mapas públicos favoritos dos que non é colaboradora.

Xestión de marcadores

RF-16. Creación de marcadores. O sistema debe permitir engadir marcadores ao mapa segundo os permisos da usuaria sobre o mesmo, permitindo indicar nome, descrición, categoría e enderezo para cada marcador.

RF-17. Modificación de marcadores. O sistema debe permitir editar a información asociada aos marcadores segundo os permisos da usuaria sobre o mapa.

RF-18. Eliminación de marcadores. O sistema debe permitir eliminar marcadores segundo os permisos da usuaria sobre o mapa.

Sistema de categorías

RF-19. Clasificación por categorías. O sistema debe permitir a asignación dunha categoría a cada punto de interese.

RF-20. Personalización visual das categorías. O sistema debe permitir definir atributos visuais das categorías, como cor ou icona.

RF-21. Filtrado por categorías. O sistema debe permitir consultar e filtrar os marcadores segundo a categoría.

Preferencias de visualización

RF-22. Activación e desactivación de capas. O sistema debe permitirlles ás persoas usuarias activar ou desactivar a visualización de determinadas categorías no mapa.

RF-23. Gardado do estado de visualización. O sistema debe permitir gardar as preferencias de visibilidade das capas para cada usuaria. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

Notas en marcadores

RF-24. Creación de notas. O sistema debe permitir engadir anotacións ou comentarios asociados a marcadores do mapa. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

RF-25. Control de visibilidade das notas. O sistema debe permitir definir se unha nota é privada ou visible para outras usuarias do mapa. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

Historial de cambios

RF-26. Rexistro de accións. O sistema debe rexistrar automaticamente as accións realizadas sobre os mapas e os seus elementos.

RF-27. Consulta do historial. O sistema debe permitir consultar o historial de cambios segundo criterios como usuaria, data ou tipo de acción.

RF-28. Restauración de versións. O sistema debe permitir restaurar versións anteriores dos elementos modificados. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

Funcionalidade offline

RF-29. Acceso sen conexión. O sistema debe permitir acceder á información previamente almacenada no dispositivo mesmo sen conexión a internet. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

2.3. Requisitos non funcionais

Seguridade e privacidade

RNF-01. Protección de credenciais. O sistema debe almacenar os contrasinais das usuarias utilizando algoritmos de hash seguros.

RNF-02. Comunicación segura. O sistema debe garantir que todas as comunicacións entre cliente e servidor se realicen mediante conexións cifradas.

RNF-03. Protección de datos persoais. O sistema debe garantir a protección da información persoal das usuarias conforme á normativa vixente en materia de protección de datos.

RNF-04. Control de acceso. O sistema debe implementar mecanismos de control de acceso que restrinxan as accións das usuarias segundo os seus permisos.

RNF-05. Minimización de datos. O sistema debe evitar o almacenamento permanente de información innecesaria ou potencialmente sensible relacionada coa actividade das usuarias.

RNF-06. Ausencia de rastreadores externos. O sistema non debe empregar cookies de terceiros nin mecanismos de rastrexo externos que poidan comprometer a privacidade das persoas usuarias.

Usabilidade

RNF-07. Interface intuitiva. A aplicación debe proporcionar unha interface clara e comprensible que facilite o uso das funcionalidades principais sen requirir coñecementos técnicos avanzados.

RNF-08. Deseño responsivo. A interface debe adaptarse correctamente a diferentes tamaños de pantalla, incluíndo dispositivos móbiles e ordenadores de escritorio.

RNF-09. WCAG. A interface debe seguir recomendacións básicas das pautas WCAG para garantir a accesibilidade.

RNF-10. Navegación eficiente. O sistema debe permitir unha navegación fluída entre as diferentes seccións da aplicación.

RNF-11. Soporte multilingüe. A aplicación debe permitir a utilización da interface en diferentes idiomas.

Rendemento

RNF-12. Tempo de resposta da API. Non debería superar os 2 segundos en condicións normais de uso.

RNF-13. Carga eficiente de datos cartográficos. O sistema debe optimizar a carga de mapas e elementos xeográficos para minimizar o tempo de visualización.

RNF-14. Xestión eficiente do almacenamento no cliente. O sistema debe xestionar de forma eficiente os datos almacenados no cliente, incluíndo as preferencias de usuaria

persistidas en localStorage, o access token mantido en memoria de módulo JavaScript durante a sesión e o refresh token almacenado en cookie HttpOnly, evitando un uso innecesario dos recursos do dispositivo.

Escalabilidade e mantemento

RNF-15. Arquitectura modular. O sistema debe estar estruturado nunha arquitectura modular que facilite a evolución e ampliación da aplicación.

RNF-16. Separación entre cliente e servidor. A aplicación debe manter unha separación clara entre o frontend e o backend mediante unha API definida.

RNF-17. Calidade do código. O código do sistema debe seguir boas prácticas de desenvolvemento que faciliten a súa comprensión, mantemento e evolución.

RNF-18. Control de versións. O desenvolvemento do proxecto debe realizarse utilizando sistemas de control de versións para garantir o seguimento dos cambios.

Compatibilidade

RNF-19. Compatibilidade con navegadores modernos. A aplicación debe funcionar correctamente nos principais navegadores web modernos que implementen os estándares actuais da web.

RNF-20. Compatibilidade con estándares cartográficos abertos. O sistema debe ser compatible cos estándares utilizados polos mapas base de OpenStreetMap e tecnoloxías relacionadas.

RNF-21. Independencia de servizos propietarios. A aplicación debe evitar a dependencia de APIs ou servizos propietarios sempre que existan alternativas abertas equivalentes.

2.3. Análise de usuaria

Os tipos de usuarias que poden interactuar coa aplicación distínguense en dous niveis. Por unha banda, existen roles a nivel global que definen o tipo de usuaria dentro do sistema. Por outra banda, establécense roles específicos asociados a cada mapa colaborativo, que permiten asignar permisos concretos aos usuarias participantes.

Esta separación permite aplicar un control de acceso máis flexible, xa que unha mesma usuaria pode ter diferentes niveis de permiso segundo o mapa no que participe.

Tipos de usuaria a nivel de sistema

1. Visitante (usuaria anónima)

Usuaria que accede á aplicación sen autenticación.

Capacidades principais:

- Consultar mapas públicos.
- Navegar polo contido cartográfico dispoñible.
- Visualizar marcadores e categorías visibles.

Non pode crear nin modificar contido.

2. Usuaria rexistrada

Usuaria que dispón dunha conta no sistema e accede mediante autenticación.

Capacidades principais:

- Crear e xestionar mapas propios.
- Consultar mapas públicos.
- Participar en mapas colaborativos aos que foi convidada.

3. Administradora da aplicación

Usuaria con privilexios administrativos a nivel global.

Capacidades principais:

- Supervisar o funcionamento da aplicación.
- Xestionar usuarias.
- Intervir en mapas en caso de problemas ou incidencias.

O rol de administradora está definido a nivel de sistema pero as funcionalidades asociadas non están implementadas na versión actual. Queda reservado para versións futuras tal como se xustifica na sección 3.3.

Tipos de usuarias a nivel de mapa

1. Membro

Usuaria con permisos básicos de consulta.

Capacidades:

- Visualizar o mapa.
- Consultar marcadores e categorías.

2. Colaboradora

Usuaria con permisos de edición sobre o contido do mapa.

Capacidades:

- Engadir marcadores.
- Modificar marcadores existentes.
- Engadir notas sobre os marcadores. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

3. Administradora do mapa

Usuaria responsable da xestión do mapa colaborativo.

Capacidades:

- Modificar configuracións do mapa.
- Xestionar membros e permisos.
- Eliminar marcadores.
- Restaurar versións anteriores mediante o historial. *Funcionalidade identificada no alcance pero non implementada na versión actual do proxecto.*

2.4. Casos de uso

CU-01 Visualizar mapas públicos

Actores: Visitante, usuaria rexistrada.

Descrición: Permite consultar mapas configurados como públicos sen necesidade de autenticación.

Precondicións: O mapa debe estar configurado como público.

Fluxo principal:

1. A usuaria accede á lista de mapas públicos.
2. O sistema amosa os mapas dispoñibles.
3. A usuaria selecciona un mapa.
4. O sistema carga o mapa coas capas e marcadores asociados.

Resultado: A usuaria visualiza o mapa e pode interactuar cos elementos visibles.

CU-02 Crear mapa

Actor: Usuaria rexistrada.

Descrición: Permite crear un novo mapa configurable pola usuaria.

Precondicións: A usuaria debe estar autenticada.

Fluxo principal:

1. A usuaria accede á opción de creación de mapas.
2. Introduce os datos do mapa (nome, descrición, visibilidade, localización).
3. O sistema valida a información.
4. O sistema crea o novo mapa e gárdao na base de datos.

Resultado: O mapa queda asociado á usuaria creadora.

CU-03 Xestionar permisos dun mapa

Actor: Administradora do mapa.

Descrición: Permite xestionar os permisos dos usuarios que participan nun mapa colaborativo.

Precondicións: A usuaria debe ser administradora do mapa.

Fluxo principal:

1. A administradora accede á sección de xestión de membros.
2. Selecciona unha usuaria existente ou envía un convite.
3. Define o rol correspondente (membro, colaboradora, administradora).
4. O sistema garda os cambios.

Resultado: Os permisos da usuaria quedan actualizados.

CU-04 Rexistro e autenticación

Actor: Visitante.

Descrición: Permite a creación dunha conta de usuaria e o acceso posterior ao sistema mediante autenticación.

Fluxo principal:

1. A persoa visitante accede á páxina de rexistro.
2. Introduce os datos necesarios para crear unha conta.
3. O sistema valida a información introducida.
4. O sistema crea a conta nun estado non verificado e envía un correo electrónico cun token de activación de uso único e expiración de 24 horas.
5. A persoa visitante accede á ligazón recibida por correo e o sistema activa a conta.
6. A usuaria pode iniciar sesión na aplicación.

Resultado: Creación da conta e acceso ás funcionalidades da aplicación.

CU-05 Edición colaborativa de mapas

Actores: Colaboradora, Administradora do mapa.

Descrición: Permite modificar o contido dun mapa colaborativo segundo os permisos asignados.

Precondicións:

- A usuaria debe estar autenticada.
- A usuaria debe ter permisos de edición no mapa.

Fluxo principal:

1. A usuaria accede ao mapa colaborativo.
2. Selecciona un elemento do mapa ou crea un novo.
3. Realiza as modificacións necesarias.
4. O sistema valida os cambios.
5. O sistema garda a modificación e rexistra a acción no historial.

Resultado: Os cambios quedan almacenados e visibles para os demais usuarias con acceso ao mapa.

CU-06 Engadir marcador

Actores: Colaboradora, Administradora do mapa.

Descrición: Permite engadir un novo marcador nun mapa colaborativo, especificando a súa localización xeográfica e a categoría asociada.

Precondicións:

- A usuaria debe estar autenticada no sistema.
- A usuaria debe ter permisos de edición no mapa correspondente.
- O mapa debe estar cargado na interface de visualización.

Fluxo principal:

1. A usuaria accede a un mapa no que ten permisos de edición.
2. A usuaria selecciona a opción de engadir marcador na interface do mapa.
3. A usuaria indica a localización do marcador (premendo sobre o mapa).
4. O sistema solicita os datos do marcador (nome, categoría, descrición opcional ou outros atributos).
5. A usuaria introduce a información requirida.
6. O sistema valida os datos introducidos.
7. O sistema garda o marcador na base de datos asociado o marcador ao mapa e á categoría seleccionada.
8. O sistema rexistra a acción no historial de cambios do mapa.

Resultado: O novo marcador aparece representado no mapa e pasa a estar dispoñible para todas as usuarias con acceso ao mapa segundo os permisos establecidos.

CU-07 Filtrar marcadores por categoría

Actores: Visitante, usuaria rexistrada, Membro, Colaboradora, Administradora do mapa.

Descrición: Permite amosar ou ocultar no mapa os marcadores pertencentes a determinadas categorías para facilitar a exploración e visualización da información.

Precondicións: A usuaria debe estar visualizando un mapa que conteña marcadores asociados a categorías.

Fluxo principal:

1. A usuaria accede a un mapa na aplicación.
2. O sistema amosa as categorías dispoñibles asociadas aos marcadores do mapa.
3. A usuaria selecciona unha ou varias categorías para activar ou desactivar a súa visualización.
4. O sistema actualiza a visualización do mapa en tempo real.
5. Os marcadores pertencentes ás categorías desactivadas deixan de amosarse no mapa.

Resultado: O mapa amósase filtrado segundo as categorías seleccionadas, permitíndole á usuaria visualizar un subconxunto específico de marcadores.

3. Alcance

O proxecto consiste no deseño, desenvolvemento e despregamento dunha aplicación web colaborativa para a creación, edición e xestión de mapas personalizados. A aplicación estará baseada exclusivamente en tecnoloxías libres e terá como principio fundamental a protección da privacidade das persoas usuarias e o control sobre os seus datos.

O sistema desenvolverase seguindo unha arquitectura cliente-servidor, composta por un *frontend* web interactivo e un *backend* encargado da lóxica de negocio, autenticación e persistencia dos datos. A comunicación entre ambos compoñentes realizarase mediante unha API REST segura. O sistema empregará unha base de datos relacional con soporte xeoespacial para o almacenamento de información cartográfica.

O alcance do proxecto inclúe tanto o desenvolvemento funcional da aplicación como o deseño da arquitectura técnica, o modelado de datos, a implementación da API e a preparación dun entorno de execución que permita o despregamento do sistema.

3.1. Funcionalidades incluídas

Xestión de usuarias e autenticación

O sistema permitirá a creación e xestión de contas de usuaria mediante un mecanismo de rexistro e autenticación seguro.

Entre as funcionalidades incluídas atópanse:

- Rexistro de novos usuarias.
- Inicio de sesión mediante autenticación baseada en *tokens*.
- Xestión do perfil de usuaria.
- Sistema de roles que determina os permisos de acceso aos recursos da aplicación.

Sistema de mapas colaborativos

A aplicación permitirá a creación e xestión de mapas personalizados que poden ser utilizados de forma individual ou colaborativa.

As funcionalidades principais inclúen:

- Creación de mapas públicos ou privados.
- Xestión da información básica de cada mapa (nome, descrición e configuración).
- Sistema de permisos que permite asignar diferentes niveis de acceso aos participantes dun mapa.
- Posibilidade de convidar outros usuarios a mapas privados mediante enlaces de acceso con caducidade.

Edición de contido xeoespacial

Os usuarios con permisos adecuados poderán modificar o contido dun mapa mediante a edición de elementos xeográficos.

Entre as accións dispoñibles inclúense:

- Engadir marcadores mediante coordenadas xeográficas.
- Modificar ou eliminar elementos existentes.
- Organizar os puntos mediante categorías configurables.
- Asociar atributos visuais aos elementos.

Historial de cambios

O sistema incorporará un mecanismo de seguimento que permita rexistrar as modificacións realizadas sobre os mapas colaborativos.

Este historial permitirá:

- Consultar accións realizadas polos usuarios.
- Identificar cambios segundo data, usuaria e tipo de operación.

Preferencias e personalización

Cada usuaria poderá configurar determinados aspectos da visualización da aplicación.

Entre as opcións incluídas atópanse:

- Activar ou desactivar capas ou categorías do mapa.
- Acceder rapidamente a mapas recentes ou favoritos.

3.2. Aspectos técnicos incluídos no proxecto

Ademais das funcionalidades descritas, o proxecto inclúe o desenvolvemento da infraestrutura técnica necesaria para o funcionamento da aplicación:

- Implementación dun *backend* baseado nun framework de desenvolvemento web para Java.
- Desenvolvemento dun *frontend* web interactivo baseado en tecnoloxías modernas de JavaScript.
- Deseño dunha API REST para a comunicación entre cliente e servidor.
- Modelado dunha base de datos relacional con soporte para datos xeoespaciais.
- Implementación de medidas de seguridade e protección da privacidade dos usuarios.

3.3. Limitacións do alcance

Determinadas funcionalidades identificadas durante a fase de análise quedaron fóra do alcance da implementación actual por razóns de viabilidade temporal e complexidade técnica, priorizando aquelas funcionalidades que constitúen o núcleo colaborativo da aplicación. A continuación descríbense estas limitacións, xunto coa súa xustificación.

Notas en marcadores

Aínda que os requisitos RF-24 e RF-25 contemplan a posibilidade de engadir anotacións ou comentarios asociados a marcadores con control de visibilidade, esta funcionalidade non foi implementada na versión actual. Os marcadores xa incorporan un campo de descrición que cobre a necesidade básica de asociar información textual a un punto de

interese, polo que se considerou que a implementación dunha capa de notas independente, con dobre nivel de visibilidade (privada ou compartida) e un modelo de datos propio, non era prioritaria respecto a outras funcionalidades do núcleo colaborativo. Queda identificada como liña de mellora para versións posteriores.

Restauración de versións no historial de cambios

O sistema rexistra automaticamente as accións realizadas sobre os mapas e os seus elementos (RF-26 e RF-27), e permite consultar o historial con filtros por tipo de acción e por usuario. Non obstante, a restauración de versións anteriores dos elementos modificados (RF-28) non está implementada. Esta funcionalidade implica, ademais da lóxica de reversión, unha estratexia de almacenamento do estado previo de cada elemento en cada acción, o que supón un alcance técnico significativamente maior. A capacidade de consulta do historial cobre o caso de uso principal de trazabilidade; a reversión queda como ampliación futura.

Funcionalidade offline

A integración de acceso sen conexión (RF-29) mediante Service Workers e almacenamento local con IndexedDB non foi implementada nesta versión. A funcionalidade *offline* require un deseño específico de sincronización bidireccional de datos, resolución de conflitos en caso de edicións concorrentes e xestión do ciclo de vida da caché, o que a converte na funcionalidade de maior complexidade técnica do proxecto. A súa implementación completa e robusta non é compatible co período de desenvolvemento dispoñible. A arquitectura da aplicación, baseada nunha SPA con API REST stateless, non presenta obstáculos estruturais para incorporar esta funcionalidade en versións futuras.

Capacidades de administración global

O sistema inclúe o rol ADMIN a nivel de aplicación, mais non se implementaron os endpoints nin a interface de administración global. As funcionalidades propias deste rol (xestión de contas de usuario, supervisión de mapas públicos e intervención en caso de incidencias) non se alían coas funcionalidades colaborativas principais do proxecto e foron descartadas para esta versión en favor delas. Queda reservada para versións futuras, onde poida deseñarse de forma coherente con requisitos de seguridade e usabilidade axeitados.

Consultas xeoespaciais nativas

As coordenadas dos marcadores e mapas almacénanse como valores numéricos de latitude e lonxitude. Aínda que o proxecto declara a dependencia de PostGIS e emprega PostgreSQL como base de datos de produción, a busca por radio utiliza a fórmula de Haversine calculada en memoria na capa de servizo Java, en lugar de delegar esta operación en consultas espaciais nativas de PostgreSQL. Esta decisión permite un desenvolvemento máis áxil durante o prototipo, pero introduce limitacións de rendemento con volumes grandes de datos. A migración a consultas PostGIS nativas queda identificada como mellora técnica para versións posteriores.

Paxinación de listaxes

Ningunha das listaxes da aplicación (mapas, marcadores, categorías, historial de cambios ou membros) implementa paxinación. Todos os resultados se cargan nun único request. Esta decisión simplifica o desenvolvemento durante o período de prototipado, pero limita a escalabilidade da aplicación ante volumes de datos elevados. A incorporación de paxinación ou carga por demanda queda como mellora técnica pendente.

4. Planificación

4.1. Metodoloxía de desenvolvemento

Para organizar o desenvolvemento do proxecto utilízase unha metodoloxía de traballo áxil baseada nos principios de Scrum, adaptada ás características dun proxecto individual de fin de ciclo.

Scrum é un marco de traballo orientado ao desenvolvemento iterativo e incremental, no que o produto evoluciona progresivamente mediante entregas parciais que permiten validar as funcionalidades implementadas e detectar posibles problemas de forma temperá.

A aplicación deste enfoque permite dividir o desenvolvemento en iteracións controladas, facilitar a planificación do traballo e manter unha visión clara do progreso do proxecto ao longo do tempo.

Metodoloxía áxil adaptada

A metodoloxía empregada baséase nunha versión simplificada de Scrum, estruturada en sprints ou iteracións de desenvolvemento, cada unha cun conxunto de tarefas definidas, un período temporal concreto e obxectivos funcionais claros.

Para organizar o traballo utilizáronse os principais artefactos de Scrum:

- **Product Backlog**, que recolle o conxunto de funcionalidades e requisitos do sistema, organizados en forma de historias de usuaria.
- **Sprint Backlog**, que contén as tarefas seleccionadas para cada iteración de desenvolvemento.
- **Incremento de produto**, correspondente ás funcionalidades implementadas e operativas ao final de cada sprint.

Este enfoque permitiu:

- Dividir o proxecto en partes asumibles e organizadas.
- Priorizar as funcionalidades máis importantes do sistema.
- Avaliar o progreso de forma continua ao longo do desenvolvemento.

- Introducir melloras ou modificacións de maneira controlada.
- Manter o foco nos obxectivos principais do proxecto.

Cada sprint incluíu tarefas de análise, deseño, implementación, probas e documentación, permitindo producir incrementos funcionais da aplicación en cada iteración.

Seguimento e control do progreso

Ao finalizar cada sprint realizouse unha revisión do traballo desenvolvido co obxectivo de comprobar o cumprimento das funcionalidades planificadas, identificar posibles problemas ou dificultades técnicas e actualizar e reorganizar o backlog do proxecto para as seguintes iteracións.

Este proceso de revisión continua permitiu manter unha visión clara do estado do proxecto e asegurar que o desenvolvemento se mantivese aliñado cos obxectivos definidos inicialmente.

4.2. Organización en sprints

O desenvolvemento do proxecto estruturouse en iteracións curtas denominadas sprints, cada unha cunha duración aproximada dunha semana. Esta duración seleccionouse para adaptarse ao calendario académico e permitir unha evolución progresiva do sistema mediante incrementos funcionais frecuentes.

Cada sprint inclúe tarefas de análise, deseño, implementación, probas e documentación. Ao finalizar cada iteración obtense un prototipo funcional do sistema, que incorpora as novas funcionalidades desenvolvidas durante ese período.

Este enfoque permite validar progresivamente o comportamento da aplicación e detectar posibles problemas de integración entre os diferentes compoñentes do sistema.

4.3. Planificación detallada dos sprints

A planificación estrutúrase en trece sprints, incluíndo un sprint inicial de preparación e análise.

Sprint 0 — Análise e preparación

Data de inicio: 02/03/2026.

Data de fin: 08/03/2026.

Obxectivo: Establecer as bases conceptuais e técnicas do proxecto, definindo a arquitectura do sistema, os requisitos iniciais e a planificación do desenvolvemento.

Tarefas:

- Definición do alcance do proxecto.
- Análise dos requisitos funcionais e non funcionais.
- Definición da metodoloxía de traballo.
- Modelado conceptual do sistema.
- Deseño preliminar da arquitectura cliente-servidor.
- Análise de viabilidade técnica das tecnoloxías seleccionadas.
- Preparación da infraestrutura inicial do proxecto.

Entregables:

- Primeiro borrador dos cinco primeiros apartados da memoria.
- Diagrama Entidade Relación.
- Diagrama de clases.
- Wireframes da aplicación.
- Guía de estilos.

Sprint 1 — Infraestrutura inicial

Data de inicio: 09/03/2026.

Data de fin: 15/03/2026.

Obxectivo: Preparar o entorno de desenvolvemento e a estrutura básica da aplicación.

Tarefas:

- Configuración dos entornos de traballo de desenvolvemento e produción.
- Configuración do *backend* en Spring Boot.

- Configuración do *frontend* en React.
- Configuración da base de datos PostgreSQL.
- Integración inicial con PostGIS.
- Estrutura inicial da API REST.

Entregables: Prototipo inicial da aplicación.

Sprint 2 — Sistema de usuarias

Data de inicio: 16/03/2026.

Data de fin: 22/03/2026.

Obxectivo: Implementar o sistema de rexistro e autenticación de usuarias.

Tarefas:

- Modelado das entidades de usuaria.
- Implementación do rexistro de usuarias.
- Implementación do inicio de sesión.
- Configuración de Spring Security.
- Implementación de autenticación mediante JWT.
- Creación dos primeiros endpoints da API de usuarias.
- Interface básica de rexistro e login no *frontend*.

Entregables: Prototipo con autenticación funcional.

Sprint 3 — Sistema de mapas

Data de inicio: 23/03/2026.

Data de fin: 29/03/2026.

Obxectivo: Implementar a estrutura básica de mapas colaborativos.

Tarefas:

- Modelado da entidade mapa.
- Creación de mapas.
- Edición e eliminación de mapas.
- Definición de visibilidade pública/privada.
- Implementación dos endpoints de xestión de mapas.
- Integración de Leaflet para visualización cartográfica.

Entregables:

- Prototipo con sistema de mapas funcional.

Sprint 4 — Sistema de marcadores

Data de inicio: 30/03/2026.

Data de fin: 05/04/2026.

Obxectivo: Permitir a creación e edición de elementos xeográficos dentro dun mapa.

Tarefas:

- Modelado de marcadores.
- Creación de endpoints para marcadores.
- Integración con PostGIS.
- Interface para engadir marcadores no mapa.
- Edición e eliminación de marcadores.

Entregables: Prototipo con edición de contido xeoespacial.

Sprint 5 — Categorías e atributos visuais

Data de inicio: 06/04/2026.

Data de fin: 12/04/2026.

Obxectivo: Organizar os puntos mediante categorías configurables.

Tarefas:

- Modelado da entidade categoría.
- Asociación de categorías a marcadores.
- Configuración de cores e iconas.
- Implementación de filtros por categoría.
- Interface de xestión de categorías.

Entregables: Prototipo con categorización de contido.

Sprint 6 — Sistema de permisos

Data de inicio: 13/04/2026.

Data de fin: 19/04/2026.

Obxectivo: Implementar o modelo de permisos para mapas colaborativos.

Tarefas:

- Modelado de roles por mapa.
- Implementación de relación usuaria-mapa.
- Control de acceso aos endpoints.
- Interface de xestión de membros.
- Implementación de convites con *token*.

Entregables: Prototipo con colaboración entre usuarias

Sprint 7 — Historial de cambios

Data de inicio: 20/04/2026.

Data de fin: 26/04/2026.

Obxectivo: Implementar o sistema de seguimento de cambios en mapas, marcadores e categorías.

Tarefas:

- Modelado da entidade historial.
- Rexistro automático de accións.
- Consulta do historial.
- Interface de visualización do historial.

Entregables: Prototipo con trazabilidade de modificacións.

Sprint 8 — Personalización do usuaria

Data de inicio: 27/04/2026.

Data de fin: 03/05/2026.

Obxectivo: Implementar preferencias de visualización e personalización.

Tarefas:

- Modelado de preferencias.
- Filtros de visibilidade de categorías.
- Gardado de configuración de usuaria.
- Interface de configuración.

Entregables: Prototipo con personalización do mapa.

Sprint 9 — Mellora da interface e rendemento

Data de inicio: 04/05/2026.

Data de fin: 10/05/2026.

Obxectivo: Mellorar a usabilidade e optimizar o rendemento do sistema.

Tarefas:

- Optimización de consultas xeoespaciais.
- Mellora da interface de usuaria.
- Optimización da carga de datos.
- Corrección de erros detectados.

Entregables: Prototipo estable da aplicación.

Sprint 10 — Funcionalidade *offline*

Data de inicio: 11/05/2026.

Data de fin: 17/05/2026.

Obxectivo: Implementar soporte para uso sen conexión.

Tarefas:

- Implementación de Service Workers.
- Almacenamento local mediante IndexedDB.
- Sistema de descarga de datos.
- Sincronización ao recuperar conexión.

Entregables: Prototipo con soporte *offline*.

Sprint 11 — Probas e estabilización

Data de inicio: 18/05/2026.

Data de fin: 24/05/2026.

Obxectivo: Validar o funcionamento global da aplicación e corrixir erros.

Tarefas:

- Probas funcionais.
- Probas de seguridade.

- Corrección de erros.
- Optimización final do sistema.

Entregables: Versión estable da aplicación.

Sprint 12 — Documentación e despregamento

Data de inicio: 25/05/2026.

Data de fin: 31/05/2026.

Obxectivo: Preparar a documentación final do proxecto e realizar o despregamento da aplicación.

Tarefas:

- Redacción da memoria técnica.
- Preparación da documentación da API.
- Configuración de Docker.
- Preparación do despregamento.
- Elaboración da presentación final.

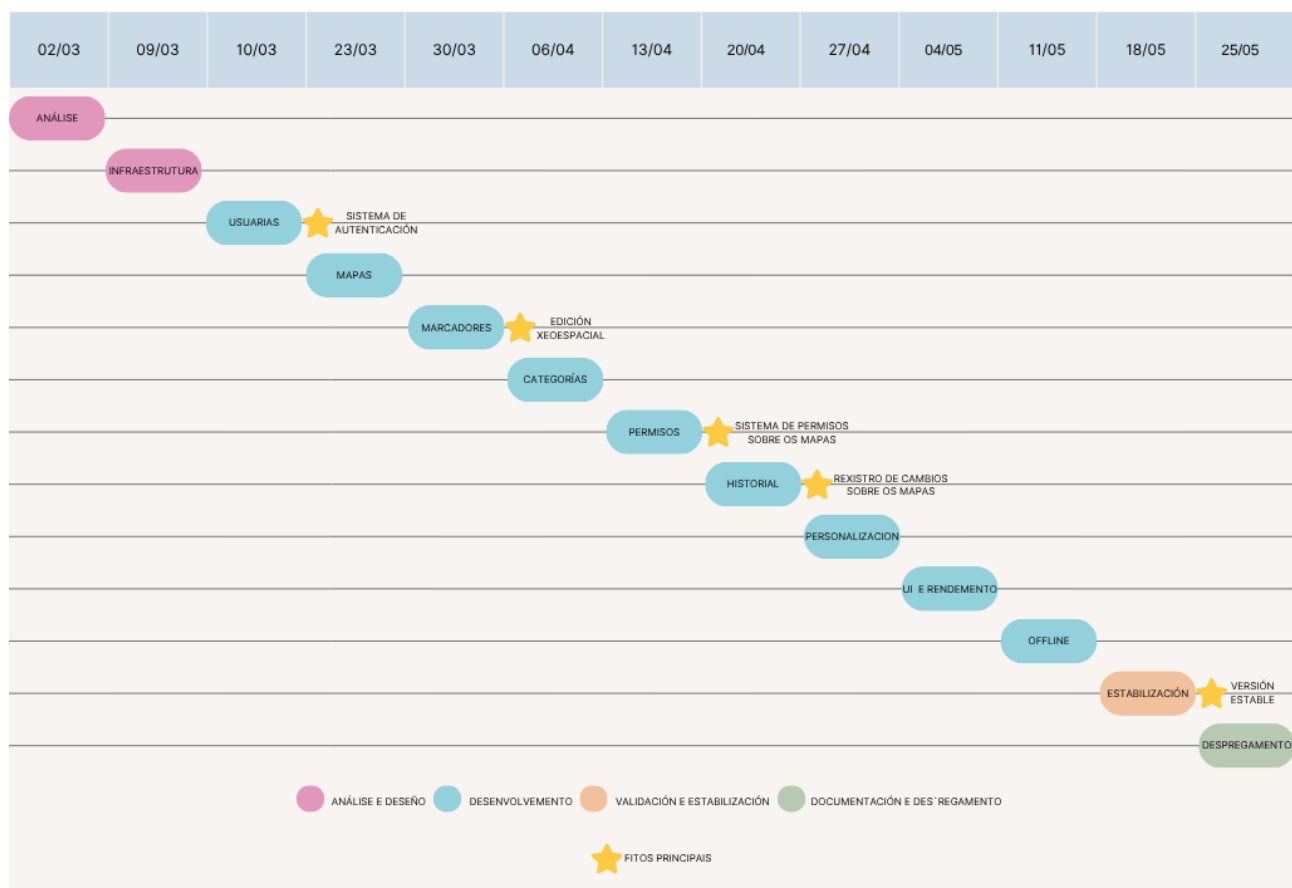
Entregables:

- Versión final da aplicación.
- Manual técnico.
- Manual de usuaria.
- Instrucións de despregamento do sistema.
- Versión final da memoria.

4.4. Fitos do proxecto

Fito	Descrición	Sprint
Definición da arquitectura	Establecemento da arquitectura do sistema e planificación inicial	Sprint 0
Infraestrutura base	Configuración do entorno de desenvolvemento e estrutura inicial da aplicación	Sprint 1
Sistema de autenticación	Implementación do rexistro e inicio de sesión de usuarias	Sprint 2
Sistema de mapas	Creación e visualización de mapas colaborativos	Sprint 3
Edición xeoespacial	Implementación de marcadores e edición de contido	Sprint 4
Sistema de permisos	Xestión de roles e colaboración entre usuarias	Sprint 6
Sistema de historial	Rexistro de cambios e trazabilidade	Sprint 7
Funcionalidade <i>offline</i>	Implementación de acceso sen conexión	Sprint 10
Versión estable	Finalización das probas e estabilización do sistema	Sprint 11
Versión final	Documentación e despregamento do sistema	Sprint 12

4.5. Diagrama de Gantt



5. Medios a utilizar

5.1. Tecnoloxías

O desenvolvemento da aplicación baséase nun conxunto de tecnoloxías amplamente utilizadas no desenvolvemento de aplicacións web modernas. A selección realizouse atendendo a criterios de robustez, seguridade, madurez do ecosistema e compatibilidade cos principios de software libre e respecto pola privacidade das persoas usuarias.

A arquitectura do sistema segue un modelo cliente-servidor, no que o backend expón unha API REST encargada da lóxica de negocio e da xestión dos datos, mentres que o frontend proporciona unha interface web interactiva que permite visualizar e editar mapas colaborativos.

Backend

Java (OpenJDK). Linguaxe principal empregada para o desenvolvemento do backend. Escollida pola súa robustez, estabilidade e amplo ecosistema para a construción de aplicacións empresariais escalables.

Spring Framework + Spring Boot. Framework e módulo utilizados para o desenvolvemento da aplicación backend e da API REST. Permiten estruturar a aplicación en compoñentes modulares e facilitan a configuración e integración de distintos servizos.

Spring Security. Módulo de seguridade do ecosistema Spring empregado para implementar autenticación, autorización e control de acceso aos recursos da aplicación.

JWT (JSON Web Tokens). Tecnoloxía empregada para a autenticación baseada en tokens entre cliente e servidor, permitindo un modelo de comunicación sen estado (*stateless*).

Hibernate / JPA. Tecnoloxía de mapeo obxecto-relacional (ORM) utilizada para facilitar a persistencia de datos e a interacción entre os obxectos da aplicación e a base de datos relacional.

PostgreSQL. Sistema de xestión de bases de datos relacional de código aberto recoñecido pola súa estabilidade, rendemento e capacidade para xestionar grandes volumes de información.

PostGIS. Extensión xeoespacial de PostgreSQL incluída na infraestrutura de produción. Na versión actual as coordenadas almacénanse como valores numéricos de latitude e lonxitude, e o filtrado por radio usa a fórmula de Haversine na capa de servizo Java. A migración a tipos xeométricos nativos e consultas ST_DWithin queda identificada como mellora técnica futura.

Frontend

React. Biblioteca JavaScript para a construción de interfaces de usuaria baseadas en compoñentes reutilizables, que permite desenvolver aplicacións web dinámicas e interactivas.

Leaflet.js. Biblioteca JavaScript lixeira para a visualización de mapas interactivos no navegador. Permite integrar mapas base de OpenStreetMap e representar elementos xeográficos como marcadores ou capas cartográficas.

OSRM (Open Source Routing Machine). Servizo de cálculo de rutas de código aberto utilizado para xerar itinerarios entre dous puntos xeográficos sobre o mapa. A integración realízase mediante fetch directo á API pública de OSRM e representación da ruta con L.geoJSON de Leaflet. Non require ningún token nin chave de autenticación, en coherencia co principio de independencia de servizos propietarios do proxecto.

Fetch API / Axios. Ferramentas utilizadas para realizar comunicacións HTTP asincrónicas entre o frontend e a API REST do backend.

Protocolos e seguridade

HTTPS. Protocolo de comunicación cifrado empregado para garantir a confidencialidade e integridade das comunicacións entre cliente e servidor.

Tecnoloxías previstas non implementadas

Tecnoloxías para soporte offline

Service Workers. Tecnoloxía do navegador que permite interceptar peticións de rede e implementar mecanismos de cache e funcionamento offline.

IndexedDB. Sistema de almacenamento local do navegador que permite gardar datos estruturados no cliente, utilizado para almacenar información necesaria cando a aplicación se utiliza sen conexión a internet.

5.2. Ferramentas

Durante o desenvolvemento do proxecto empregáronse diversas ferramentas de software destinadas a facilitar a implementación, organización e documentación da aplicación.

Desenvolvemento e control de versións

Git. Sistema de control de versións distribuído utilizado para rexistrar os cambios realizados no código fonte e facilitar o seguimento do desenvolvemento do proxecto.

GitHub. Plataforma de hospedaxe de repositorios Git empregada para almacenar o código do proxecto e xestionar as distintas versións do desenvolvemento.

Documentación e probas da API

Swagger / OpenAPI. Ferramentas utilizadas para documentar e probar os endpoints da API REST, facilitando a comprensión da interface de programación e a interacción entre cliente e servidor.

Despregamento

Docker. Tecnoloxía de contedorización empregada para empacar a aplicación e as súas dependencias, permitindo executar o sistema en diferentes entornos de maneira consistente.

Deseño e modelado

Draw.io. Ferramenta utilizada para a elaboración de diagramas técnicos, incluíndo diagramas UML, modelos de arquitectura e diagramas de fluxo.

Moqups. Ferramenta empregada para a creación de prototipos de interface e o deseño de *mockups* da aplicación.

Canva. Ferramenta de deseño gráfico empregada como complemento para a elaboración da guía de estilos do proxecto.

Claude Design. Asistente de intelixencia artificial empregado como ferramenta auxiliar para o deseño de mockups da interface e a elaboración de materiais para a presentación final.

6. Presuposto

O proxecto foi desenvolvido priorizando o uso de software libre e infraestrutura de provedores europeos aliñados cos principios de privacidade e independencia tecnolóxica do proxecto. Os custos directos limítanse ao aloxamento do servidor e ao rexistro do dominio; todas as ferramentas de desenvolvemento e a maioría dos servizos de soporte son de uso libre e gratuíto.

6.1. Custos de infraestrutura

A infraestrutura de produción compónse dun único servidor en Hetzner e o rexistro do dominio en Gandi. Ambos os provedores foron escollidos por criterios de independencia europea, cumprimento do RGPD e valores aliñados cos do proxecto.

Concepto	Detalle	Custo
Servidor Hetzner CX22	2 vCPUs, 4 GB RAM, 40 GB disco, 20 TB tráfico. Facturación mensual sen compromiso de permanencia.	3,79€/mes
Dominio explora-mapa.eu	Rexistrado en Gandi por un ano. Extensión .eu, provedor europeo con política de privacidade pública.	3,99€/ano
Certificado SSL/TLS	Xestionado con Let's Encrypt e Certbot. Renovación automática.	0,00€
Total infraestrutura	Para o período do proxecto (1 mes de servidor + 1 ano de dominio)	7,78€

6.2. Custos de ferramentas e servizos

Todas as ferramentas de desenvolvemento, control de versións e documentación utilizadas no proxecto son de uso libre e gratuíto, sen custos de licenza.

Ferramenta	Custo
IntelliJ IDEA Ultimate	0,00€
Visual Studio Code	0,00€
GitHub	0,00€
Docker / Docker Compose	0,00€
Postal (servidor de correo)	0,00€
Brevo (relay SMTP saínte)	0,00€
Mailpit (SMTP local dev)	0,00€
Draw.io (diagramas)	0,00€
Canva (guía de estilos)	0,00€
Total ferramentas	0,00€

6.3. Custo de desenvolvemento

O proxecto foi desenvolvido por unha única persoa ao longo de aproximadamente 14 semanas, desde finais de febreiro de 2026 ata a entrega en xuño de 2026. Estímanse 175 horas de traballo efectivo distribuídas entre análise, deseño, implementación, probas e documentación.

Para a valoración económica aplícase o salario bruto de referencia dun perfil de desenvolvemento web junior en España segundo os convenios colectivos do sector TIC:

Concepto	Horas	Custo
Análise e deseño (Sprints 0–1)	30 h	360,00€
Implementación backend (Sprints 2–8)	55 h	660,00€
Implementación frontend (Sprints 3–9)	65 h	780,00€
Probas, despregamento e documentación (Sprints 11–12)	25 h	300,00€
Total desenvolvemento	175 h	2.100,00€

6.4. Resumo de custos totais

Partida	Importe
Infraestrutura (servidor + dominio)	7,78€
Ferramentas e licenzas	0,00€
Desenvolvemento (175 h × €12/h)	2.100,00€
TOTAL	2.107,78€

O custo total estimado do proxecto é de €2.107,78, dos cales o 99,6% corresponde ao traballo de desenvolvemento. O custo de infraestrutura é marxinal grazas ao uso de ferramentas gratuítas e provedores de baixo custo aliñados cos valores do proxecto.

7. Execución

7.1. Deseño do sistema

Arquitectura xeral da aplicación

A aplicación segue unha arquitectura cliente-servidor con separación completa entre o frontend e o backend, comunicados mediante unha API REST sobre HTTP. Esta separación permite que ambos os compoñentes evolucionen de forma independente e facilita o despregamento en contornos distintos.

O **backend** é unha aplicación Java construída con Spring Boot que expón unha API REST stateless. Xestiona a autenticación, a lóxica de negocio, o control de acceso e a persistencia dos datos. En desenvolvemento utiliza unha base de datos H2 en memoria; en produción, PostgreSQL.

O **frontend** é unha Single Page Application (SPA) construída con React e Vite. Comunícase co backend exclusivamente a través da API REST, xestiona o estado da sesión en memoria e renderiza a interface de xeito dinámico sen recargar a páxina.

Modelo de datos

O modelo de datos xira arredor de tres entidades principais: Usuario, Mapa e Marcador, complementadas por entidades de relación e soporte.

Entidades principais

Usuario representa unha persoa rexistrada no sistema. Almacena as credenciais de acceso (contrasinal como hash bcrypt), o rol de aplicación (USER ou ADMIN), o idioma preferido e o estado de verificación do correo. Está relacionada cos mapas que crea, os mapas nos que participa como membro, os mapas que garda como favoritos e os convites enviados e recibidos.

Mapa é o elemento central da aplicación. Cada mapa ten un nome, unha descrición opcional, coordenadas xeográficas de referencia (latitude e lonxitude), datos de localización obtidos por xeolocalización inversa (cidade, rexión, país), e un tipo de visibilidade (PUBLICO ou PRIVADO). O campo creadoPor almacena o nome de usuaria da propietaria e determina os permisos máximos sobre o mapa.

Marcador representa un punto de interese dentro dun mapa. Ten nome, descrición opcional, coordenadas xeográficas propias, e pode estar asociado a unha categoría. O campo creadoPor permite aplicar a lóxica de permisos baseada no rol da usuaria dentro do mapa.

Entidades de relación

MapaMembro establece a relación entre unha usuaria e un mapa privado, almacenando o rol asignado (MEMBRO, COLABORADORA ou ADMIN_MAPA). É a entidade que habilita a colaboración estruturada.

Convite xestiona o proceso de incorporación a un mapa privado. Contén un token UUID único, as referencias á usuaria anfitroia e á convidada, o mapa de destino, o rol que se lle asignará ao aceptar, o estado do convite e a data de expiración. A expiración é *lazy*: o convite non se marca automaticamente como EXPIRADO por ningún proceso de fondo, senón no momento en que se intenta usar ou consultar.

MapaGardado implementa o sistema de favoritos: relaciona unha usuaria cun mapa público que desexa gardar para acceso rápido, sen implicar permisos de edición.

EntradaHistorial rexistra as accións realizadas sobre os elementos dun mapa. Almacena o tipo de acción (CREAR, EDITAR, ELIMINAR), o tipo de elemento afectado (MAPA, MARCADOR, CATEGORIA), o identificador e nome do elemento, o nome de usuaria que realizou a acción, e un campo de detalle con información adicional. A usuaria almacénase como String (non como FK) para preservar o rexistro histórico aínda que a conta sexa eliminada.

Categoría permite organizar os marcadores dun mapa en grupos visuais, con nome, cor en formato hexadecimal e icona. A eliminación dunha categoría desvincula os marcadores asociados sen eliminalos.

Xestión da autenticación

RefreshToken almacena o hash do token de refresco activo de cada sesión, coa data de expiración e un indicador de revogación. O token real nunca se persiste; só o seu hash, comparado no momento da renovación.

TokenVerificacion xestiona os tokens de activación de conta enviados por correo electrónico, con data de expiración e indicador de uso.

Relacións e integridade referencial

A eliminación dun Mapa propaga en cascada a todos os seus elementos dependentes: categorías, marcadores, membros, gardados e convites. O `EntradaHistorial` elimínase mediante unha query `@Modifying JPQL` explícita, xa que a súa relación co mapa non usa cascade JPA estándar. A eliminación dunha `Usuaria` require un proceso secuencial que limpa primeiro as súas participacións en mapas alleos antes de eliminar os seus propios mapas.

Deseño da capa de servizo e lóxica de negocio

O backend segue un patrón de capas estrito: os `Controller` reciben e validan as peticións HTTP e delegan en `Service`; os `Service` conteñen toda a lóxica de negocio e de acceso; os `Repository` proporcionan acceso á base de datos mediante Spring Data JPA.

O control de acceso non usa anotacións `@PreAuthorize` nin roles de Spring Security nos endpoints. En cambio, realízase na capa de servizo mediante `MapaAccesoService`, un compoñente utilitario que centraliza a verificación de acceso e é reutilizado por `CategoriaServiceImpl`, `MarcadorServiceImpl`, `HistorialServiceImpl` e `MapaMembroServiceImpl`. Esta decisión mantén a lóxica de permisos nun único punto e evita duplicación entre servizos.

Os `Service` comunican erros ao `Controller` mediante excepcións semánticas: `IllegalArgumentException` para recursos non atopados (mapeada a HTTP 404) e `IllegalStateException` para operacións non permitidas (mapeada a HTTP 403). O `GlobalExceptionHandler (@RestControllerAdvice)` captura estas excepcións e formata respostas de erro consistentes en JSON.

O mapeo entre entidades e DTOs realízase manualmente nos `Service`, sen uso de ferramentas de mapeo automático como `MapStruct`. Os `Request DTO` están anotados con Jakarta Validation para a validación de entrada; os `Response DTO` garanten que nunca se expón ningunha entidade JPA directamente ao cliente.

Deseño da interface de usuaria

A interface está organizada como unha SPA con dúas zonas diferenciadas: páxinas públicas (login, rexistro, verificación e mapa principal) e páxinas protexidas que requiren autenticación, accesibles tras pasar polo compoñente `ProtectedRoute`.

As páxinas protexidas comparten o `AppLayout`, que inclúe un `Sidebar` lateral con navegación principal, expandible e colapsable, e un `BottomNav` para dispositivos móbiles.

A interface adapta os elementos dispoñibles segundo o rol da usuaria sobre cada mapa: o rol efectivo calcúlase no cliente comparando o campo creadoPor do mapa co nome de usuaria en sesión, e completando coa entrada en MapaMembro se existe.

A visualización cartográfica usa Leaflet con tiles de OpenStreetMap. O compoñente MapViewer opera en dous modos: só lectura (cando onLocationSelect é null) e modo de selección de localización (cando se está a crear ou editar un marcador). Un pin provisional mostra a posición seleccionada antes de confirmar. A busca de enderezos usa a API de Nominatim con debounce de 400 ms para limitar as peticións.

O estado global da aplicación xestiónase con dous mecanismos complementarios: AuthContext (React Context) para o estado de sesión, e cinco stores Zustand para estado de UI persistente (sidebar, idioma, tema visual) e estado transitorio (notificacións, visualización de mapas activos).

A interface soporta modo claro e escuro mediante un atributo data-theme no elemento raíz <html>. As cores de ambos os modos están definidas como propiedades CSS personalizadas en global.css, o que permite o cambio de tema sen recargar a páxina. A preferencia gárdase e xestiónase mediante useTemaStore.

Deseño da seguridade e control de acceso

A autenticación baséase en JWT con dous tokens: un accessToken de curta duración (15 minutos en desenvolvemento, 5 minutos por defecto) e un refreshToken de 7 días con rotación. Cada vez que se usa o refreshToken para renovar a sesión, o token antigo revógase e xérase un novo par, o que limita a ventá de uso en caso de compromiso.

Os accessToken almacénanse en memoria no frontend (variables de módulo en axiosInstance.js), sen persistencia en localStorage nin cookies, o que os protexe de ataques XSS. Os refreshToken envíanse ao cliente como cookie HttpOnly, Secure e SameSite=Strict, o que impide o acceso desde JavaScript e os protexe de ataques XSS. No servidor almacénase unicamente o hash SHA-256 do token real, polo que o valor orixinal nunca queda rexistrado de forma recuperable.

O ciclo de vida da sesión no frontend xestiónase mediante un interceptor de Axios: cada petición inxecta o accessToken na cabeceira Authorization; en caso de resposta 401, o interceptor intenta un refresco silencioso e, se falla, emite un evento DOM session-expired que propaga a redirección ao login sen interacción da usuaria.

Os contrasinais almacénanse con hash bcrypt mediante BCryptPasswordEncoder de Spring Security. As comunicacións deben realizarse sobre HTTPS en produción para garantir a confidencialidade dos tokens en tránsito.

7.2. Implementación

Tecnoloxías utilizadas

O backend implementouse en Java 25 con Spring Boot 4.0.4, usando Spring MVC para a capa HTTP, Spring Security para autenticación e autorización, Spring Data JPA con Hibernate como ORM, e jjwt 0.13.0 para a xeración e validación de JWT. O envío de correos electrónicos xestiónase con JavaMailSender; en desenvolvemento úsase Mailpit como servidor SMTP local. A base de datos de desenvolvemento é H2 en memoria; a de produción, PostgreSQL con a dependencia postgres-jdbc preparada para uso xeoespacial futuro. Lombok elimina boilerplate de getters, setters e construtores.

O frontend implementouse en React 19.2.4 con Vite 8.0.1 como ferramenta de build. O enrutamento é client-side con React Router 7. As peticións HTTP usan Axios con interceptores para a xestión automática de tokens. A visualización cartográfica usa Leaflet 1.9 con tiles de OpenStreetMap. O estado global xestiónase con Zustand 5. A internacionalización usa react-i18next con ficheiros de tradución JSON en galego e inglés, cargados en runtime mediante i18next-http-backend.

Estrutura do código

O backend organízase no paquete raíz `explora.map`, dividido en subcapas: `controller`, `service` (interfaces e implementacións), `repository`, `entity`, `dto`, `security` e `config`. Toda a nomenclatura (paquetes, clases, métodos e campos) está en galego, o que mantén coherencia co idioma do proxecto.

O frontend organízase en `src/pages` (unha páxina por ruta), `src/components` (compoñentes reutilizables), `src/services` (funcións de chamada á API, un ficheiro por recurso), `src/store` (AuthContext e stores Zustand), `src/hooks` e `src/assets/styles` (CSS global por módulo). Os estilos usan clases globais BEM con variables CSS definidas en `global.css`; non se usan CSS Modules.

Integración de compoñentes e servizos externos

Nominatim (OpenStreetMap): ao crear ou editar un mapa, `MapaServiceImpl` chama a <https://nominatim.openstreetmap.org/reverse> coas coordenadas do mapa para obter a localización en linguaxe natural (cidade, rexión, país, código de país). O parseo da

resposta JSON é manual. Se a chamada falla, os campos de localización quedan a null sen interromper a operación principal. Tamén se usa desde o frontend para a busca de localizacións por texto libre en MapSearchBar.

OpenStreetMap / Leaflet: o frontend carga os tiles cartográficos directamente desde os servidores de OpenStreetMap a través de Leaflet. Non se require ningún token nin API key, en coherencia co principio de independencia de servizos propietarios do proxecto.

Open-Meteo: o frontend consulta a API pública de <https://api.open-meteo.com> para obter datos meteorolóxicos en tempo real (temperatura, estado do ceo, vento) asociados á localización seleccionada no mapa principal. É unha API libre, sen chave de autenticación, que non rexistra nin vende datos de uso.

Correo electrónico: en desenvolvemento, os correos de verificación de conta envíanse a un servidor Mailpit local que os captura sen reenvialos, permitindo probar o fluxo completo de rexistro sen servidor SMTP real. En produción o sistema de correo está implementado con Postal como servidor de xestión e seguimento, con envío saínte enrutado a través de Brevo como relay SMTP, dado que Hetzner bloquea o porto 25 por defecto en contas novas. Esta configuración é transparente para a aplicación: Spring Boot envía ao servidor SMTP de Brevo usando as credenciais do relay.

Infraestrutura de produción: a aplicación está despregada nun servidor privado virtual de Hetzner Online GmbH, empresa alemá independente con centros de datos en Alemaña e Finlandia, cumprimento RGPD e alimentación con enerxías renovables. O despregamento realízase mediante Docker Compose, con tres servizos contenerizados: base de datos PostgreSQL con extensión PostGIS, backend Spring Boot e frontend servido por Nginx. O dominio explora-mapa.eu está rexistrado en Gandi, rexistrador francés con código de conduta público orientado á privacidade.

Descrición das principais funcionalidades implementadas

Sistema de autenticación e verificación de conta. O rexistro crea a conta nun estado non verificado e envía un correo con un token de activación de uso único e expiración de 24 horas. Ata que a conta non se verifica, o login é rexeitado. Este fluxo protexe contra a creación masiva de contas con correos falsos. Un useRef guard no compoñente VerificarPage evita que o modo estrito de React consuma o token dúas veces na montaxe do compoñente.

Sistema de mapas colaborativos con roles. Cada mapa privado ten asociada unha táboa de membros (MapaMembro) con rol individual por usuario. O proceso de incorporación ocorre mediante convites: a propietaria ou unha administradora do mapa

envía un convite a outra usuaria, que pode aceptalo ou rexeitalo. Ao aceptar, créase a entrada en MapaMembro co rol especificado no convite. Os roles determinan os permisos de forma granular: MEMBRO só pode ler; COLABORADORA pode crear elementos e editar ou eliminar os que ela mesma creou; ADMIN_MAPA pode editar e eliminar calquera elemento e xestionar membros. A propietaria ten sempre permisos totais independentemente da táboa de membros.

Historial de cambios. Cada operación de creación, edición ou eliminación sobre mapas, marcadores e categorías xera automaticamente unha EntradaHistorial. O historial é consultable con filtros por tipo de acción e por nome de usuaria. A usuaria almacénase como String no historial para que os rexistros persistan aínda que a conta da usuaria sexa eliminada posteriormente.

Explorador de mapas públicos. A páxina principal permite buscar mapas públicos por proximidade xeográfica. O backend filtra os mapas públicos usando a fórmula de Haversine calculada en Java, comparando as coordenadas de cada mapa co punto de busca. A busca de lugares usa Nominatim no frontend con debounce de 400 ms. Os mapas resultado pódense gardar como favoritos para acceso rápido desde a lapela "Gardados" da listaxe de mapas.

Internacionalización. A interface soporta galego e inglés. O idioma détéctase na orde: preferencia gardada en localStorage, e como último recurso, o idioma do navegador. A preferencia sincronízase co perfil da usuaria no backend para persistir entre dispositivos. O cambio de idioma é instantáneo sen recargar a páxina.

Eliminación de conta. A eliminación execútase nunha única transacción que limpa en orde os tokens de sesión, os gardados, as membresías en mapas alleos, os convites (como anfitriño e como convidada), os mapas propios con todo o seu contido en cascada, e finalmente a entidade de usuaria. O historial de accións da usuaria en mapas alleos pervive, identificado co nome de usuaria como texto, para non comprometer a trazabilidade de eses mapas.

7.3. Probas e validación

Estratexia de probas

A estratexia de probas do proxecto combina dous niveis complementarios: probas unitarias illadas sobre compoñentes de lóxica pura, e probas de integración que exercitan o sistema de punta a punta desde a capa HTTP ata a base de datos.

As probas unitarias aplícanse sobre clases e funcións con lóxica suficientemente complexa e illable, onde a verificación sen dependencias externas aporta valor real. As probas de integración cobren os fluxos completos do sistema, en particular a lóxica de control de acceso por rol, que é o aspecto máis crítico desde o punto de vista da seguridade e o máis difícil de verificar manualmente de forma sistemática.

A selección dos casos de test priorizou a cobertura dos escenarios de maior risco: autenticación, validación de tokens, control de acceso a recursos e diferenciación de comportamento entre roles.

Probas unitarias

Implementáronse tres suites de tests unitarios puros: dúas no backend, sen levantar o contexto de Spring, e unha no frontend, sen montar compoñentes React.

JwtUtilsTest.java — Xeración e validación de tokens JWT (3 tests)

Verifica de forma illada a lóxica de JwtUtils, instanciando a clase directamente con new JwtUtils() e inxectando os valores de configuración mediante ReflectionTestUtils.setField():

Test	Escenario	Resultado esperado
T1	Token xerado é válido e contén o username correcto	ValidarToken: true obterUsernameDeToken: username correcto
T2	Token con expiración de 1ms é rexeitado tras agarda	ValidarToken: false
T3	Token con firma manipulada é rexeitado	ValidarToken: false

MapaAccesoServiceTest.java — Lóxica de control de acceso a mapas (4 tests)

Verifica de forma illada a lóxica de MapaAccesoService, mockeando MapaRepository e MapaMembroRepository con Mockito mediante @InjectMocks e @Mock, sen acceso á base de datos:

Test	Escenario	Resultado esperado
------	-----------	--------------------

T1	Mapa público accedido por calquera usuaria	Sen excepción
T2	Mapa privado accedido pola propietaria	Sen excepción
T3	Mapa privado accedido por membro con entrada en MapaMembro	Sen excepción
T4	Mapa privado accedido por usuaria foránea sen entrada	IllegalStateException

usePasswordStrength.test.js — Cálculo de forza de contrasinal (3 tests)

Verifica de forma illada a función `calcularForza` exportada desde o hook `usePasswordStrength.js`, sen montar ningún compoñente React, usando Vitest:

Test	Escenario	Resultado esperado
T1	Contrasinal curto só con minúsculas ("abc")	<code>forza === 'débil'</code> , porcentaxe ≤ 33
T2	Contrasinal con maiúsculas e números ("Abc123")	<code>forza === 'moderado'</code> , porcentaxe entre 34 e 66
T3	Contrasinal con maiúsculas, números e símbolos ("Abc123!\$")	<code>forza === 'forte'</code> , porcentaxe > 66

Probas de integración

Implementouse unha suite de 15 tests de integración no ficheiro `PermisosIntegrationTest.java`, usando `@SpringBootTest` con `MockMvc` e base de datos H2 en memoria. Cada test é independente grazas a `@Transactional` con rollback automático ao remate.

Os tests organízanse en tres grupos:

Grupo A — Autenticación e validación de JWT (4 tests)

Verifican que a capa de seguridade funciona correctamente de forma independente da lóxica de negocio:

Test	Escenario	Resultado esperado
T6	Request sen cabeceira JWT	401
T7	JWT con firma manipulada	401
T8	Login con contrasinal incorrecto	401
T9	Login con conta non verificada	403

Grupo B — Control de acceso por rol (5 tests)

Verifican que o sistema de permisos por rol (MEMBRO, COLABORADORA, ADMIN_MAPA, propietaria) se aplica correctamente sobre as operacións de escritura:

Test	Escenario	Resultado esperado
T1	Usuaría foránea accede a mapa privado	403
T2	MEMBRO intenta crear marcador	403
T3	COLABORADORA crea marcador propio	201
T4	COLABORADORA elimina marcador alíeo	403
T5	Propietaria elimina calquera marcador	204

Grupo C — Funcionalidade e casos límite (6 tests)

Verifican operacións CRUD básicas e escenarios de interacción entre roles:

Test	Escenario	Resultado esperado
T10	Propietaria crea categoría	201
T11	Propietaria lista marcadores do seu mapa	200 + array non baleiro

T12	Propietaria edita o seu mapa	200
T13	Eliminación de mapa elimina contido en cascada	204 + 404 no mapa eliminado
T14	ADMIN_MAPA edita marcador alleo	200
T15	Fluxo completo de convite: bloqueo previo, aceptación e acceso posterior	403 → 200 → 200

A execución completa da suite (mvn test) produce o seguinte resultado:

Tests run: 15, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Probos de usuaria

As probas de usuaria realizáronse de forma manual sobre a interface web con ambos servidores en execución (backend en localhost:8080, frontend en localhost:5173). Verificáronse os fluxos principais da aplicación seguindo os casos de uso definidos na sección 2.4:

- Rexistro, verificación de correo e inicio de sesión.
- Creación, edición e eliminación de mapas.
- Engadir, editar e eliminar marcadores e categorías.
- Filtrado de marcadores por categoría.
- Envío e aceptación de convites entre usuarias.
- Visualización do historial de cambios.
- Cambio de rol de membros por parte dunha administradora do mapa.
- Navegación entre as tres lapelas da lista de mapas (proprios, colaboracións, gardados).

Resultados e incidencias detectadas

A execución completa de todas as suites produce os seguintes resultados:

Backend (mvn test):

Tests run: 22, Failures: 0, Errors: 0, Skipped: 0

BUILD SUCCESS

Frontend (npm test):

Test Files 1 passed

Tests 3 passed

Durante as probas manuais de usuaria detectáronse as seguintes incidencias, corrixiadas durante o proceso de desenvolvemento:

- **Dobre consumo do token de verificación en React StrictMode:** o efecto de verificación executábase dúas veces en modo desenvolvemento, consumindo o token na primeira chamada e fallando na segunda. Resolto mediante un `guard useRef` que impide a segunda invocación.
- **Peches incorrectas de menús despregables en contedores con overflow: hidden:** os menús kebab das tarxetas de mapa quedaban recortados polo contedor pai. Resolto mediante `ReactDOM.createPortal` con coordenadas absolutas obtidas de `getBoundingClientRect`.
- **Desincronización do estado de categorías tras eliminar un marcador:** ao eliminar un marcador cunha categoría asignada, a lista de categorías non se actualizaba visualmente. Resolto elevando o estado de categorías a `MapaDetallePage` e coordinando as recargas de marcadores e categorías no mesmo callback.

7.4. Despregamento e documentación

Configuración do entorno de execución

Entorno de desenvolvemento

O entorno de desenvolvemento está deseñado para permitir un ciclo de iteración rápido sen dependencias externas. O backend execútase con Spring Boot co perfil dev activo,

que configura automaticamente unha base de datos H2 en memoria con esquema recreado en cada arranque (create-drop), o que elimina a necesidade de xestionar migracións durante o desenvolvemento. Os correos de verificación de conta son capturados por Mailpit, un servidor SMTP local que non reenvía mensaxes pero permite inspeccionar o contido dos correos desde unha interface web en <http://localhost:8025>.

O frontend execútase co servidor de desenvolvemento de Vite, que ofrece Hot Module Replacement (HMR) para reflectir os cambios no código instantaneamente no navegador sen recarga completa. A URL base do backend configúrase mediante a variable de entorno `VITE_API_URL` definida no ficheiro `.env.development`, que apunta a <http://localhost:8080> durante o desenvolvemento.

Para lanzar o entorno de desenvolvemento:

```
# Backend (IntelliJ IDEA ou terminal)

./mvnw spring-boot:run
```

```
# Frontend (VSCode ou terminal)

npm run dev
```

A aplicación queda accesible en <http://localhost:5173>.

Entorno de produción

O entorno de produción está completamente contenerizado mediante Docker Compose, o que garante a reproducibilidade do despregamento independentemente do sistema operativo do servidor. O stack principal comprende tres servizos:

- **PostgreSQL con PostGIS** (postgis/postgis:16-3.4-alpine): base de datos relacional con soporte xeoespacial. Os datos persisten nun volume Docker nomeado (postgres_data) que sobrevive aos reinicios e actualizacións dos contedores.
- **Backend** (explora-backend): imaxe construída en dúas fases — compilación con Maven e runtime con JRE Alpine. O perfil prod activa a conexión a PostgreSQL e a validación estrita do esquema (validate).
- **Frontend** (explora-frontend): imaxe construída con Node.js e servida por Nginx Alpine. Nginx actúa como servidor estático para a SPA React e como proxy inverso

para as peticións á API, enrutando `/api/` ao contedor do backend, e como proxy para o panel de administración de Postal en `postal.explora-mapa.eu`. As URLs da API configúranse mediante a variable `VITE_API_URL` definida en `.env.production`, que usa rutas relativas (`/api`) para evitar problemas de CORS en produción.

Adicionalmente, o sistema de correo electrónico execútase como un stack Docker independente en `/opt/postal/`, composto por cinco servizos:

- **Postal** (`ghcr.io/postalserver/postal:3.3.5`): servidor de correo de código aberto que xestiona o envío, o seguimento e o historial de mensaxes. Expón o panel de administración no porto 5000 (accesible via Nginx) e o servidor SMTP no porto 25.
- **MariaDB** (`mariadb:10.11`): base de datos para Postal.
- **Redis** (`redis:7-alpine`): cola de mensaxes para Postal.

Dado que Hetzner bloquea o porto 25 saínte por defecto en contas novas, o correo saínte enrutase a través de **Brevo** (anteriormente Sendinblue) como relay SMTP. Brevo ofrece un plan gratuíto de 300 correos diarios, suficiente para o volume de correos transaccionais da aplicación. Esta configuración é transparente para a aplicación: Spring Boot envía ao servidor SMTP de Brevo (`smtp-relay.brevo.com:587`) usando as credenciais do relay.

A orquestración do stack principal garante a orde correcta de arranque: PostgreSQL arranca primeiro e pasa un healthcheck antes de que o backend se inicie, e o backend debe estar en marcha antes de que o frontend estea dispoñible.

Instrucións de instalación e despregamento

Requisitos previos do servidor

- Sistema operativo: Debian 12 ou Ubuntu 22.04+
- RAM mínima: 4 GB (o stack completo con Postal require aproximadamente 3,5 GB en uso)
- Docker Engine 24+ e Docker Compose V2
- Portos 80, 443 (HTTP/HTTPS), 25 (SMTP) e 22 (SSH) accesibles
- Dominio propio con acceso ao panel DNS

Instalación de Docker en Debian 12

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl gnupg  
sudo install -m 0755 -d /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/debian/gpg  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
sudo chmod a+r /etc/apt/keyrings/docker.gpg  
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/debian $(. /etc/os-release && echo  
"$VERSION_CODENAME") stable"  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli containerd.io  
docker-buildx-plugin docker-compose-plugin
```

Despregamento inicial

```
# Clonar os repositorios  
cd /opt  
git clone https://github.com/ORGANIZACION/backend.git backend  
git clone https://github.com/ORGANIZACION/frontend.git frontend  
git clone https://github.com/ORGANIZACION/deploy.git deploy  
# Copiar e configurar as variables de entorno  
cp deploy/.env.example .env  
nano .env # Editar con valores reais  
# Copiar o docker-compose.yml  
cp deploy/docker-compose.yml .
```



```
# Lanzar o stack

docker compose up --build -d

# Verificar que os tres servizos están en marcha

docker compose ps
```

Variables de entorno requiridas (ficheiro .env)

Variable	Descrición
POSTGRES_USER	Usuario da base de datos
POSTGRES_PASSWORD	Contrasinal da base de datos
JWT_SECRET	Clave secreta para asinar tokens JWT (mín. 32 caracteres)
MAIL_HOST	Servidor SMTP para correos de verificación
MAIL_PORT	Porto SMTP (habitualmente 587)
MAIL_USERNAME	Usuario SMTP
MAIL_PASSWORD	Contrasinal SMTP
CORS_ALLOWED_ORIGINS	Orixe permitida para CORS (URL pública da app)
APP_BASE_URL	URL base pública para os enlaces de verificación por correo

Despregamento do stack de correo (Postal)

```
# Crear directorio e configuración

mkdir -p /opt/postal/config
```

```
openssl genrsa -out /opt/postal/config/signing.key 2048

# Copiar o docker-compose de Postal
cp deploy/postal-docker-compose.yml /opt/postal/docker-compose.yml

# Crear o postal.yml (ver deploy/postal.yml.example)
nano /opt/postal/config/postal.yml

# Arrancar dependencias e inicializar BD
cd /opt/postal

docker compose up -d mariadb redis

sleep 20

docker run --rm -v /opt/postal/config:/config ghcr.io/postalserver/postal:3.3.5 postal
initialize

# Arrancar o stack completo
docker compose up -d

# Crear usuaria administradora
docker exec -it postal-web postal make-user
```

O panel de administración de Postal queda accesible en <https://postal.explora-mapa.eu> tras configurar o proxy Nginx e obter o certificado SSL correspondente.

Configuración DNS

Para o correcto funcionamento do correo electrónico é necesario configurar os seguintes rexistros DNS no provedor do dominio:

Tipo	Nome	Valor	Propósito
A	@	IP do servidor	Dominio principal
A	Mail	IP do seridor	Servidor SMTP
A	Postal	IP do servidor	Servidor SMTP
MX	@	Mail.explora-mapa.eu (prioridade 10)	Panel de Postal
TXT	@	v=spf1 a mx include:spf.postal.explora-mapa.eu include:spf.brevo.com ~all	SPF
TXT	postal-XXXXXX._domainkey	Clave DKIM de Postal	DKIM para Postal
CNAME	brevo1._domainkey	b1.explora-mapa-eu.dkim.brevo.com.	DKIM para Brevo
CNAME	brevo2._domainkey	b2.explora-mapa-eu.dkim.brevo.com.	DKIM para Brevo
TXT	_dmarc	v=DMARC1; p=none; rua=mailto:rua@dmarc.brevo.com	DMARC
CNAME	psrp.explora-mapa.eu	rp.postal.example.com.	Return Path de Postal

Configuración HTTPS

Para habilitar HTTPS é necesario que o dominio xa apunte á IP do servidor e que o stack Docker estea en marcha. Os certificados SSL xestiónanse con Certbot e Let's Encrypt:

```
apt-get install -y certbot

# Certificado para o dominio principal

certbot certonly --webroot \

-w /opt/certbot/webroot \

-d explora-mapa.eu -d www.explora-mapa.eu

# Certificado para o panel de Postal

certbot certonly --webroot \

-w /opt/certbot/webroot \

-d postal.explora-mapa.eu
```

Os certificados gárdanse en `/etc/letsencrypt/live/` e son montados no contedor de Nginx como volumes de só lectura. Nginx redirixe automaticamente todo o tráfico HTTP ao porto 443 con HTTPS. Certbot configura unha tarefa programada que renova os certificados automaticamente antes da súa caducidade; os certificados de Let's Encrypt teñen unha validez de 90 días pero a renovación é transparente.

A aplicación está dispoñible publicamente en <https://explora-mapa.eu> e o panel de Postal en <https://postal.explora-mapa.eu>.

Actualización da aplicación

```
cd /opt/backend && git pull

cd /opt/frontend && git pull

docker compose up --build -d
```

Manual de usuaria

Rexistro e acceso

Para acceder á aplicación diríxete a <https://explora-mapa.eu> no navegador. Se non tes conta, preme en *Rexistrarse* e completa o formulario con nome, nome de usuaria, correo electrónico e contrasinal. Recibirás un correo de verificación; ao premer na ligazón activarase a conta e poderás iniciar sesión.



Crear conta

Comeza a crear e compartir os teus mapas personalizados.

NOME *

NOME DE USUARIA *

CORREO ELECTRÓNICO *

CONTRASINAL *

CONFIRMAR CONTRASINAL *

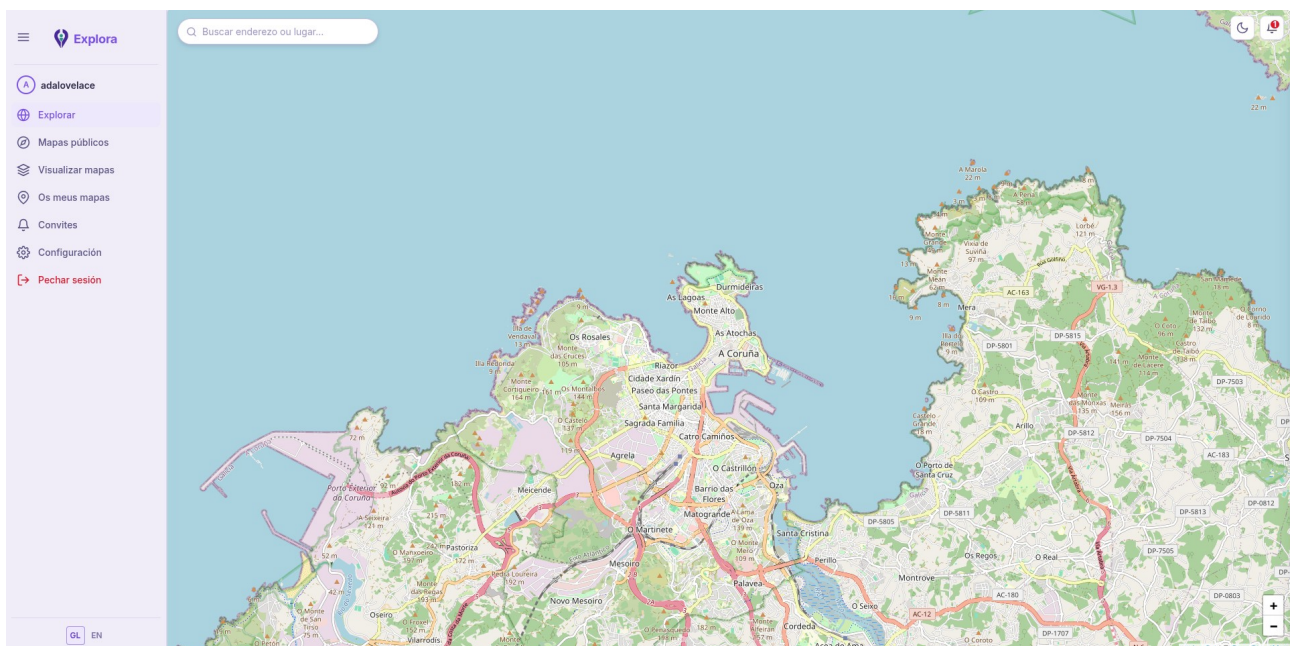
Crear conta

[¿Xa tes conta? Inicia sesión](#)

Mapa principal

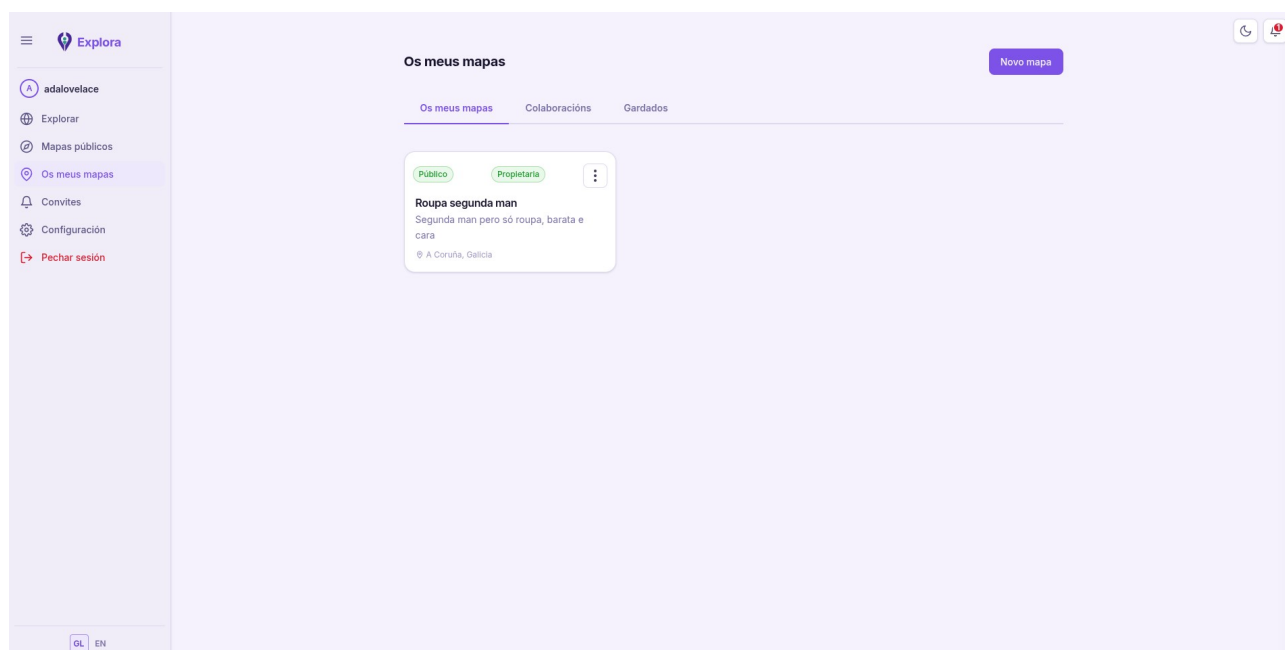
Ao iniciar sesión accedes ao mapa principal, que mostra o mapa base de OpenStreetMap. Desde aquí podes:

- Buscar calquera localización do mundo usando a barra de busca.
- Consultar a meteoroloxía actual da zona seleccionada.
- Calcular rutas entre dous puntos.
- Activar mapas propios, colaboracións ou mapas gardados para superpoñelos no mapa principal mediante o panel lateral *Visualizar mapas*.



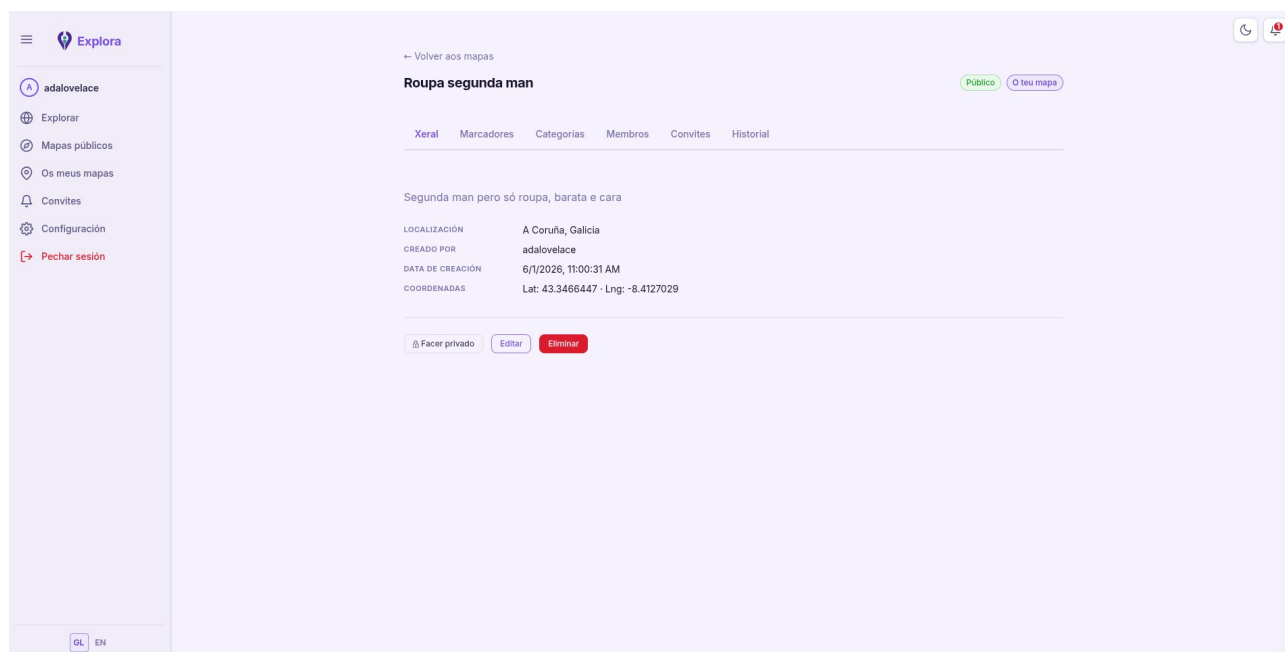
Xestión de mapas

Desde a sección *Os meus mapas* podes crear, editar e eliminar mapas propios. Cada mapa ten un nome, descrición, localización xeográfica e visibilidade (público ou privado). Os mapas públicos son visibles para calquera usuaria desde a sección *Explorar*.



Marcadores e categorías

Dentro dun mapa podes engadir marcadores premendo sobre o mapa para seleccionar a localización. Cada marcador ten nome, descrición e pode asociarse a unha categoría. As categorías permiten organizar visualmente os marcadores mediante cor e icona personalizadas, e filtrar a visualización no mapa por categoría.



The screenshot shows the 'Explora' web application interface. On the left is a sidebar with a menu containing: 'Explora', 'Mapas públicos', 'Os meus mapas', 'Convites', 'Configuración', and a 'Pegar sesión' button. The main area displays a map marker titled 'Roupa segunda man'. Above the title are buttons for 'Publico' and 'O teu mapa'. Below the title is a horizontal tab bar with 'Xeral', 'Marcadores', 'Categorías', 'Membros', 'Convites', and 'Historial'. The 'Xeral' tab is selected, showing a description: 'Segunda man pero só roupa, barata e cara'. Below this is a table with the following data:

LOCALIZACIÓN	A Coruña, Galicia
CREADO POR	adalovelace
DATA DE CREACIÓN	6/1/2026, 11:00:31 AM
COORDENADAS	Lat: 43.3466447 - Lng: -8.4127029

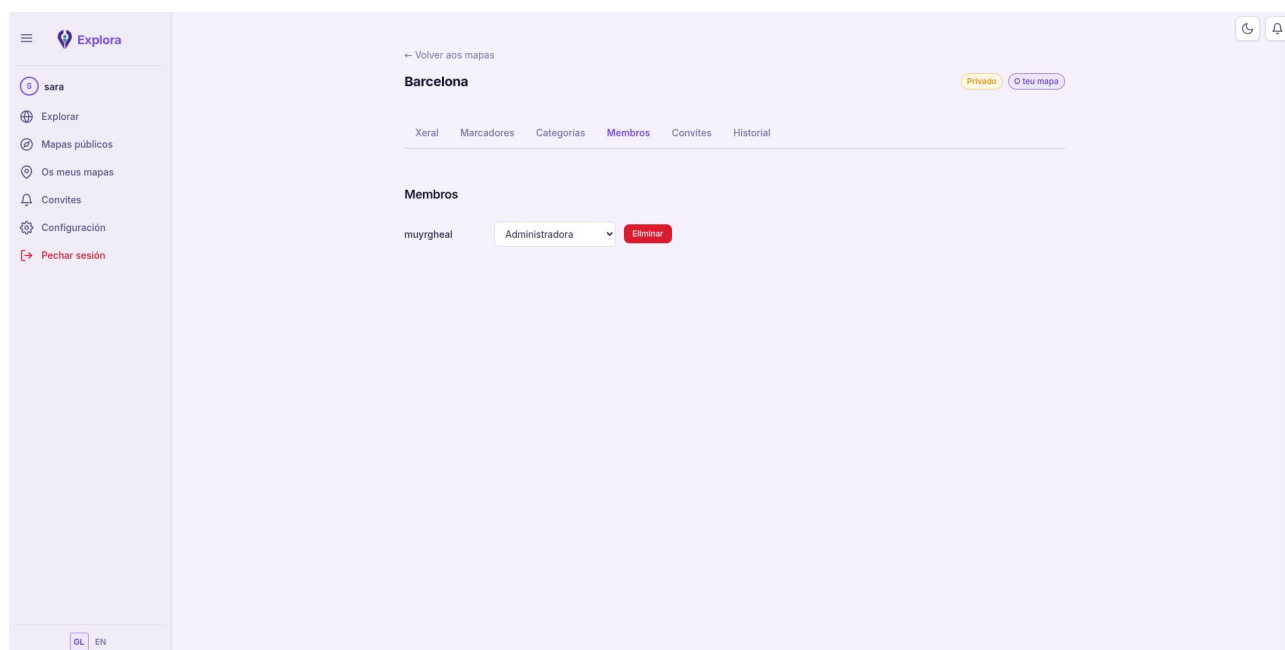
At the bottom of the marker details are three buttons: 'Facer privado', 'Editar', and 'Eliminar'. The bottom of the interface shows a language selector with 'GL' and 'EN' options.

Colaboración

Os mapas privados admiten colaboradoras mediante un sistema de convites. Desde o panel de membros podes invitar outras usuarias por nome de usuaria e asignarlles un rol:

- *Membro*: pode ver o mapa e os seus contidos.
- *Colaboradora*: pode crear e editar marcadores e categorías.
- *Administradora do mapa*: pode xestionar membros, cambiar roles e configurar o mapa.

As usuarias convidadas reciben o convite na sección *Convites* e poden aceptalo ou rexeitalo.



Historial de cambios

Cada mapa rexistra automaticamente todas as accións realizadas: creación, edición e eliminación de mapas, marcadores e categorías. O historial é consultable desde o panel *Historial* dentro de cada mapa, con filtros por tipo de elemento e por usuario.

← Volver aos mapas

Barcelona Privado O teu mapa

Xeral Marcadores Categorías Membros Convites **Historial**

Historial

Todos Marcadores Categorías

- Editou o marcador "Parc del Forum" — sara (ti) 6/7/2026, 3:53:19 PM
- Creou o marcador "Parc del Forum" — sara (ti) 6/7/2026, 3:52:33 PM
- Editou o marcador "Casa Leandro" — sara (ti) 6/7/2026, 3:51:08 PM
- Editou o marcador "Casa Leandro" — sara (ti) 6/7/2026, 3:50:45 PM
- Creou a categoría "Casa" — sara (ti) 6/7/2026, 3:49:46 PM
- Creou o marcador "Casa Leandro" — sara (ti) 6/7/2026, 3:49:14 PM
- Creou o marcador "Paral lel" — sara (ti) 6/7/2026, 3:48:24 PM
- Creou a categoría "Concertos" — sara (ti) 6/7/2026, 3:46:12 PM
- Creou o mapa "Barcelona" — sara (ti) 6/7/2026, 3:51:51 PM

OL EN

Perfil e configuración

Desde *Perfil* podes editar o teu nome, nome de usuaria, correo e contrasinal. Desde *Configuración* podes cambiar o idioma da interface entre galego e inglés. A preferencia de idioma sincronízase co servidor e mantense entre sesións.

The screenshot shows the 'O meu perfil' page. On the left is a sidebar with a menu: 'sara', 'Explorar', 'Mapas públicos', 'Os meus mapas', 'Convites', 'Configuración', and 'Pegar sesión'. The main content area is titled 'O meu perfil' and includes a user profile card with a circular avatar containing the letter 'S', the email '@sara' and 'eusonsara@protonmail.com', and icons for settings and a plus sign. Below this is a 'DATOS PERSOAIS' section with form fields for 'Nome' (filled with 'Sara'), 'Nome de usuaria' (filled with 'sara'), 'Correo electrónico' (filled with 'eusonsara@protonmail.com'), 'Contrasinal actual', 'Novo contrasinal', and 'Confirmar novo contrasinal'. Each password field has an eye icon to toggle visibility. A 'Gardar cambios' button is at the bottom right. Below the form is a 'CONVITES RECIBIDOS' section with the message 'Non hai convites pendentes.' At the bottom left of the sidebar, there are language selection buttons for 'GL' and 'EN'.

The screenshot shows the 'Configuración' page. The sidebar is identical to the previous page. The main content area is titled 'Configuración' and has two sections. The 'IDIOMA' section contains two buttons: 'Galego' (which is selected and has a checkmark) and 'English'. The 'TEMA' section contains two buttons: 'Claro' (with a sun icon) and 'Escuro' (with a moon icon). At the bottom left of the sidebar, there are language selection buttons for 'GL' and 'EN'.

Manual técnico

Visión xeral da arquitectura

Explora Map segue unha arquitectura cliente-servidor con separación estrita entre frontend e backend. O backend expón unha API REST stateless autenticada con JWT. O frontend é unha Single Page Application que consume esa API. En produción, ambos os compoñentes están contenerizados e orquestrados con Docker Compose.

Backend

O backend está construído con Spring Boot 4 sobre Java 25. A estrutura interna segue o patrón de tres capas: Controller (presentación), Service (lóxica de negocio) e Repository (persistencia con Spring Data JPA). O control de acceso por rol a nivel de mapa está centralizado en MapaAccesoService, que é inxectado polos servizos que o requiren.

A autenticación usa JWT stateless: o accessToken ten unha duración de 15 minutos e o refreshToken (almacenado como hash en base de datos) de 7 días con rotación en cada uso. O filtro AuthTokenFilter intercepta cada petición e valida o token antes de establecer o contexto de seguridade.

As excepcións de negocio (IllegalArgumentException para recursos non atopados, IllegalStateException para acceso denegado) son capturadas polo GlobalExceptionHandler e convertidas en respostas HTTP semánticas (404 e 403 respectivamente).

Frontend

O frontend está construído con React 19 e Vite 8. O enrutamento é client-side con React Router 7. O estado global de UI xestiónase con Zustand (sidebar, idioma, mapa visual, notificacións). A autenticación usa React Context, con tokens JWT almacenados en memoria de módulo JavaScript para evitar vulnerabilidades XSS asociadas ao uso de localStorage.

As chamadas á API pasan por unha instancia de Axios con interceptores que inxectan o accessToken en cada petición e xestionan o refresco silencioso cando reciben un 401.

A internacionalización está implementada con react-i18next, con ficheiros de tradución en galego e inglés cargados en runtime. A preferencia de idioma persiste en localStorage e sincronízase co backend ao cambiar.

Base de datos

O esquema é xestionado por Hibernate. En desenvolvemento úsase create-drop sobre H2 en memoria. En produción úsase validate sobre PostgreSQL, o que significa que calquera cambio no modelo de datos require unha migración manual antes de despregar. Non se usa Flyway nin Liquibase na versión actual.

Seguridade

- Contraseñais almacenados con BCrypt.
- Tokens JWT asinados con HMAC-SHA256.
- CSRF desactivado (API stateless).
- CORS configurado explicitamente para a orixe do frontend.
- Refresh tokens almacenados como hash SHA-256 en base de datos, con rotación en cada uso e revogación explícita ao pechar sesión

Documentación técnica

A documentación técnica do proxecto está publicada en liña e accesible sen autenticación nos seguintes enderezos:

Recurso	Detalles	URL
Swagger UI	Documentación interactiva da API	https://explora-mapa.eu/swagger-ui/index.html
OpenAPI JSON	Especificación da API	https://explora-mapa.eu/v3/api-docs
JavaDoc	documentación do código backend	https://explora-mapa.eu/javadoc/index.html

O Swagger UI permite explorar e probar todos os endpoints da API REST directamente desde o navegador, incluíndo os esquemas de request e response de cada operación. A especificación OpenAPI 3.1 en formato JSON está dispoñible para integrarse con xeradores de código ou ferramentas externas.

O JavaDoc cobre os ficheiros Java do backend, documentando clases, métodos, parámetros e valores de retorno. Xérase automaticamente con Maven mediante o plugin maven-javadoc-plugin e está publicado como directorio estático servido por Nginx.

7.5. Resultados e avaliación

Cumprimento dos obxectivos

A continuación analízase o grao de cumprimento dos once obxectivos específicos definidos na sección 1.2 do proxecto:

OE-01 — Integrar mapas base de OpenStreetMap: implementado mediante Leaflet 1.9 con tiles de OpenStreetMap, sen necesidade de tokens nin dependencias de APIs propietarias.

OE-02 — Mapas personalizados públicos e privados: implementado. Cada mapa ten visibilidade configurable (público/privado) que a propietaria pode cambiar en calquera momento.

OE-03 — Descubrimento de mapas por localización: implementado mediante busca por radio con fórmula de Haversine no backend e integración con Nominatim para xeolocalización inversa.

OE-04 — Sistema de convites e roles: implementado con tres roles por mapa (MEMBRO, COLABORADORA, ADMIN_MAPA) e un sistema de convites mediante token UUID con caducidade configurable.

OE-05 — Historial de cambios: implementado. Rexístranse automaticamente todas as accións de creación, edición e eliminación sobre mapas, marcadores e categorías, con filtros por tipo de elemento e por usuaria.

OE-06 — Notas en marcadores: non implementado. Identificado desde a planificación como fóra de alcance; os marcadores dispoñen dun campo de descrición como alternativa funcional.

OE-07 — Sistema de categorías con cor e icona: implementado. As categorías teñen cor hexadecimal e icona personalizables, e permiten filtrar a visualización dos marcadores no mapa.

OE-08 — Xestión de usuarias e control de acceso: implementado con rexistro, autenticación JWT, verificación de conta por correo electrónico, perfil editable e eliminación de conta.

OE-09 — Perfil de usuaria: implementado. A usuaria pode editar nome, nome de usuaria, correo, contrasinal (con verificación do contrasinal actual) e preferencia de idioma.

OE-10 — Funcionalidade offline: non implementado por viabilidade temporal. A sincronización bidireccional mediante Service Workers e IndexedDB queda identificada como liña de mellora prioritaria.

OE-11 — Seguridade e cifrado das comunicacións: implementado con HTTPS mediante Let's Encrypt, autenticación JWT stateless con access token en memoria de módulo JavaScript e refresh token en cookie HttpOnly, contrasinais con BCrypt e CORS configurado explicitamente.

Dos once obxectivos específicos, nove foron implementados completamente. Os dous non implementados (notas en marcadores e funcionalidade offline) foron identificados desde a fase de planificación como posibles limitacións de alcance e documentados na sección 3.3.

Valoración do rendemento e usabilidade

O rendemento da aplicación en produción cumpre os requisitos non funcionais establecidos. As respostas da API REST para operacións habituais (listado de mapas, carga de marcadores, autenticación) sitúanse por baixo dos 200 ms medidos en probas locais; os tempos reais dependen da latencia da rede do usuario. A carga dos tiles cartográficos de OpenStreetMap depende dos servidores de OSM e varía segundo a zona e o nivel de zoom, pero é consistente co comportamento esperado para calquera aplicación baseada en Leaflet.

Desde o punto de vista da usabilidade, a interface foi deseñada seguindo un enfoque mobile-first con deseño responsivo que se adapta a dispositivos móbiles e de escritorio. A navegación organízase mediante un sidebar en escritorio e unha barra de navegación inferior en móbil, seguindo os patróns de navegación habituais en cada tipo de dispositivo. O soporte multilingüe (galego e inglés) está implementado con i18next e a preferencia sincronízase co servidor, cumprindo os requisitos RNF-07 a RNF-11.

A seguridade cumpre os requisitos establecidos: contrasinais con BCrypt, tokens JWT en memoria de módulo JavaScript (non en localStorage, evitando vulnerabilidades XSS), refresh tokens rotativos almacenados como hash SHA-256 en base de datos, HTTPS con certificado de Let's Encrypt e ausencia total de rastreadores externos ou cookies de terceiros (RNF-01 a RNF-06).

Comparativa con solucións existentes

O proxecto sitúase nun espazo diferenciado respecto ás solucións existentes de mapas colaborativos:

Característica	Explora	Google My Maps / uMap
Software libre	Código baixo GPL-3.0	Non / Parcialmente
Autoaloxamento posible	Si (Docker)	Non / Limitado
Independencia de APIs privadas	Total (OSM, OSRM, Nominatim)	Dependencia de Google APIs
Control granular de roles por mapa	Si (3 roles)	Limitado
Historial de cambios	Si (con filtros)	Non
Verificación de conta por correo	Si	Google Account
Privacidade e RGPD	Privacy by design	Datos cedidos a terceiros
Funcionalidade offline	Non implementada	Parcial (uMap)

A principal vantaxe diferencial de Explora Map fronte a alternativas como Google My Maps é a independencia respecto das APIs de datos xeográficos: nin a aplicación nin as usuarias dependen de ningún servizo propietario para a visualización e o procesamento de información cartográfica. O código fonte está publicado baixo licenza GPL-3.0; algúns servizos de soporte empregados no despregamento, como o aloxamento, o rexistro de dominio ou o relay de correo, non son software libre en sentido estrito, aínda que foron escollidos por criterios de privacidade e independencia europea. Fronte a uMap (que tamén é software libre baseado en OSM), Explora Map ofrece un sistema de roles máis granular, historial de cambios e unha arquitectura moderna con API REST e SPA desacoplada que facilita futuras extensións.

7.6. Conclusións e liñas de mellora

Conclusións xerais

O proxecto Explora Map logrou desenvolver unha aplicación web de cartografía colaborativa funcional, despregada en produción e baseada integramente en tecnoloxías libres. Ao longo de 12 sprints e aproximadamente 14 semanas de traballo, implementáronse os bloques funcionais fundamentais: xestión de usuarias con autenticación JWT, sistema de mapas con visibilidade configurable, edición xeoespacial con marcadores e categorías, colaboración por roles con sistema de convites, historial de cambios e despregamento contenerizado en infraestrutura europea.

Desde o punto de vista técnico, o proxecto demostrou a viabilidade de construír unha aplicación web moderna con separación estrita entre frontend e backend, sen dependencias de grandes provedores tecnolóxicos, con un nivel de seguridade adecuado e un custo de infraestrutura mínimo. A elección de tecnoloxías (Spring Boot, React, PostgreSQL, Docker, OpenStreetMap) demostrou ser adecuada para o alcance do proxecto: trátase dun stack maduro e ben documentado que facilitou o avance iterativo sen introducir riscos técnicos innecesarios.

Desde o punto de vista metodolóxico, a aplicación dun proceso áxil adaptado con sprints semanais permitiu detectar problemas cedo e reaxustar prioridades en función do tempo dispoñible. A decisión de deixar fóra do alcance a funcionalidade offline e as notas en marcadores foi coherente cos recursos disponibles e non comprometeu a calidade do produto entregado.

Limitacións do proxecto

As principais limitacións da versión actual son:

1. **Funcionalidade offline non implementada:** o soporte para acceso sen conexión mediante Service Workers e IndexedDB foi identificado desde a planificación como unha funcionalidade de alta complexidade e quedou fóra do alcance da versión actual.
2. **Ausencia de paxinación:** todas as listaxes (mapas, marcadores, historial) cárganse completas nun único request. Para volumes de datos grandes isto pode xerar problemas de rendemento.
3. **Migracións de base de datos manuais:** o proxecto non usa Flyway nin Liquibase. Calquera cambio no modelo de datos en produción require intervención manual, o que é un risco operacional en contornos de longa duración.

4. **Despregamento manual sen CI/CD:** as actualizacións requiren conexión SSH ao servidor e execución de comandos manuais. Non hai pipeline automatizado de integración e entrega continua.

5. **Busca xeoespacial con Haversine en Java:** a busca de mapas por radio implementase en código Java en lugar de aproveitar as capacidades nativas de PostGIS (ST_DWithin), o que limita o rendemento en consultas xeoespaciais complexas.

6. **Cobertura de tests limitada:** os tests cobre os casos máis críticos (permisos, JWT) pero non hai tests de compoñentes React nin tests E2E.

Posibles melloras e traballos futuros

As liñas de mellora máis relevantes para versións futuras son:

1. **Funcionalidade offline:** implementar Service Workers para caché de tiles e datos, e IndexedDB para almacenamento local de mapas descargados, con sincronización bidireccional cando se recupere a conexión.

2. **Optimización con PostGIS nativo:** substituír a fórmula de Haversine en Java por consultas ST_DWithin de PostGIS para mellorar o rendemento das buscas xeoespaciais.

3. **Pipeline CI/CD:** implementar GitHub Actions para automatizar o build, as probas e o despregamento en cada push á rama principal, eliminando a necesidade de intervención manual no servidor.

4. **Migracións de base de datos:** incorporar Flyway ou Liquibase para xestionar os cambios no esquema de forma controlada e reproducible.

5. **Paxinación:** implementar paxinación con Spring Data Page nos endpoints que devolven listaxes, e carga incremental no frontend con scroll infinito ou botón de 'cargar máis'.

6. **Migración do access token a cookie HttpOnly.** Avaliar a migración do access token desde a variable de módulo JavaScript actual a unha cookie HttpOnly con protección CSRF adicional, o que eliminaría completamente a posibilidade de acceso desde JavaScript e reforzaría a resistencia a ataques XSS. Esta mellora requiriría implementar un mecanismo de protección CSRF no backend, dado que o modelo stateless actual non o contempla.

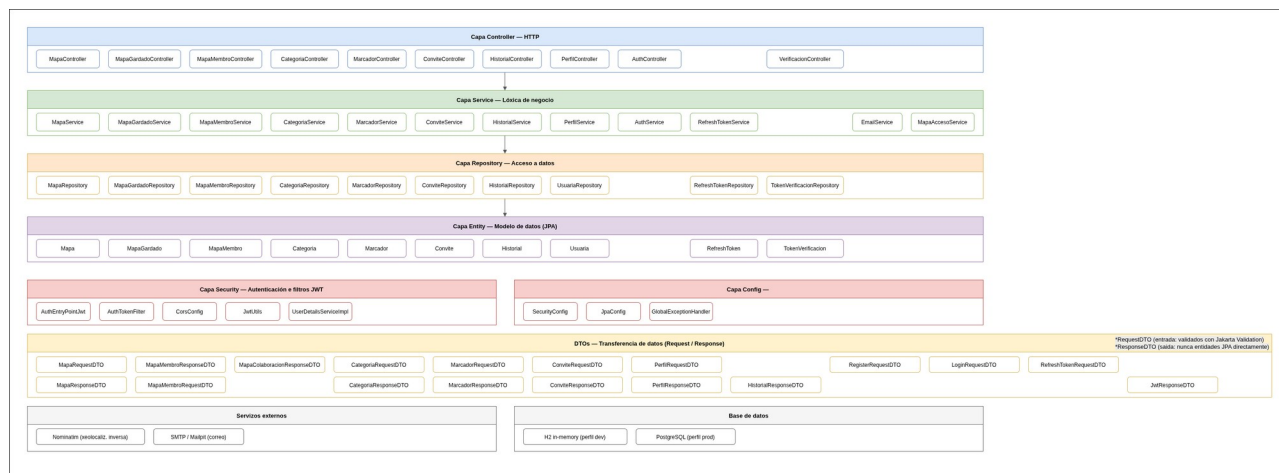
7. **Notas en marcadores:** implementar o sistema de notas independentes por marcador con control de visibilidade, identificado nos requisitos funcionais RF-24 e RF-25.

8. **Notificacións en tempo real:** substituír o polling actual (cada 60 segundos) por un sistema de notificacións push mediante WebSocket ou Server-Sent Events.

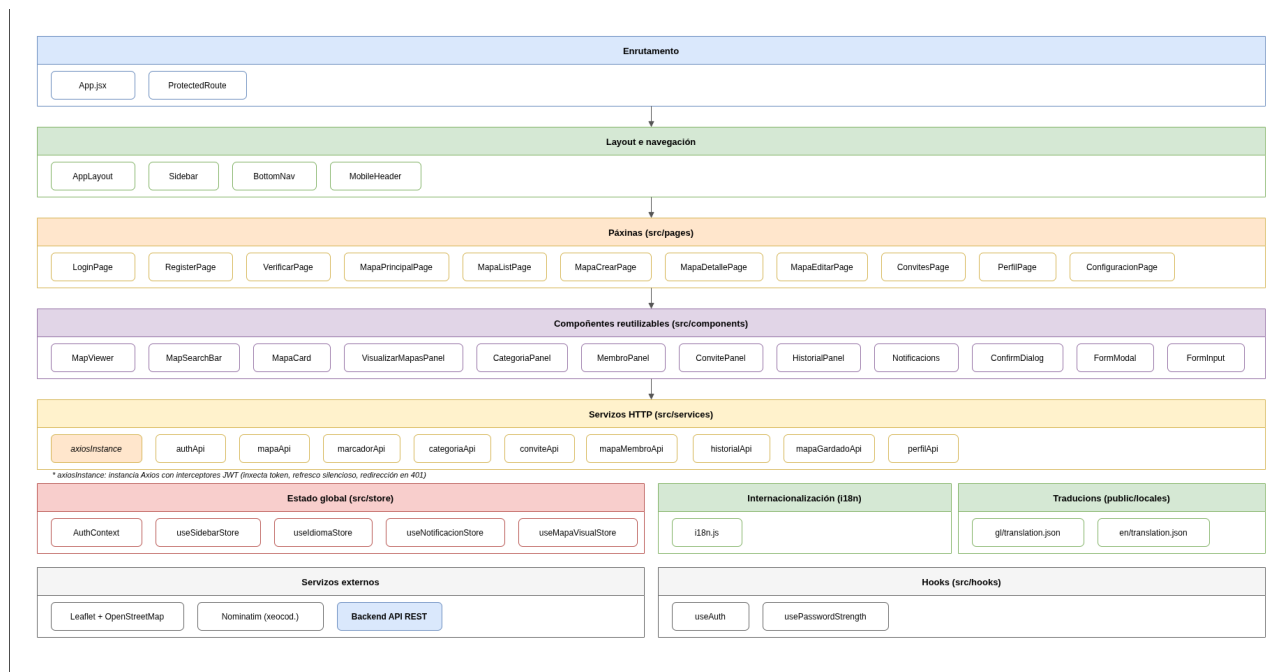
9. **Administración global:** implementar a interface de administración para o rol ADMIN de aplicación, que permita xestionar usuarias, moderar mapas públicos e consultar métricas de uso.

Anexos

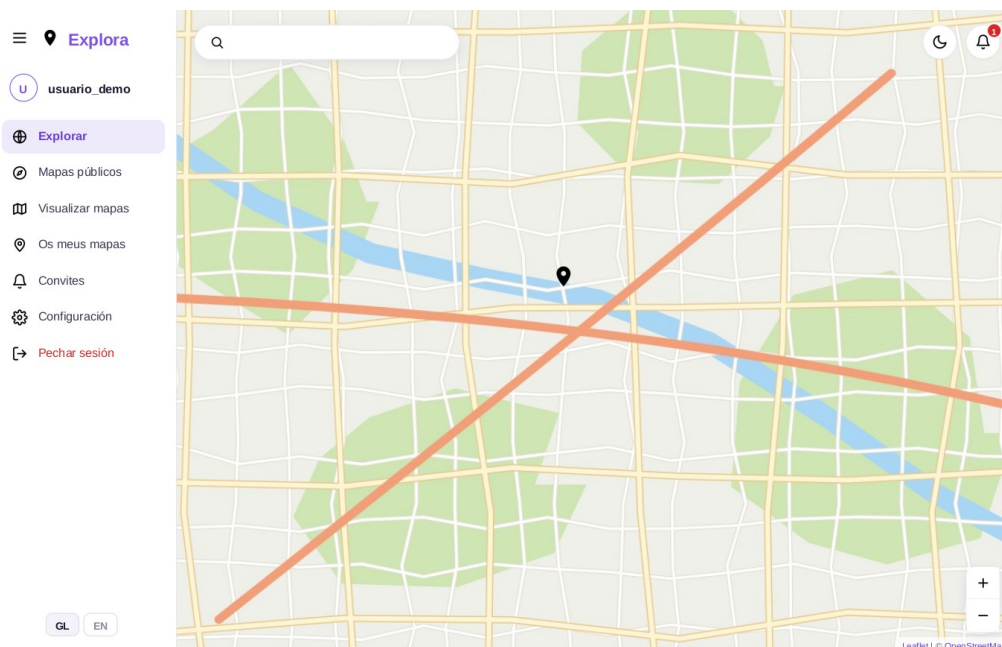
1. Diagrama da arquitectura do backend



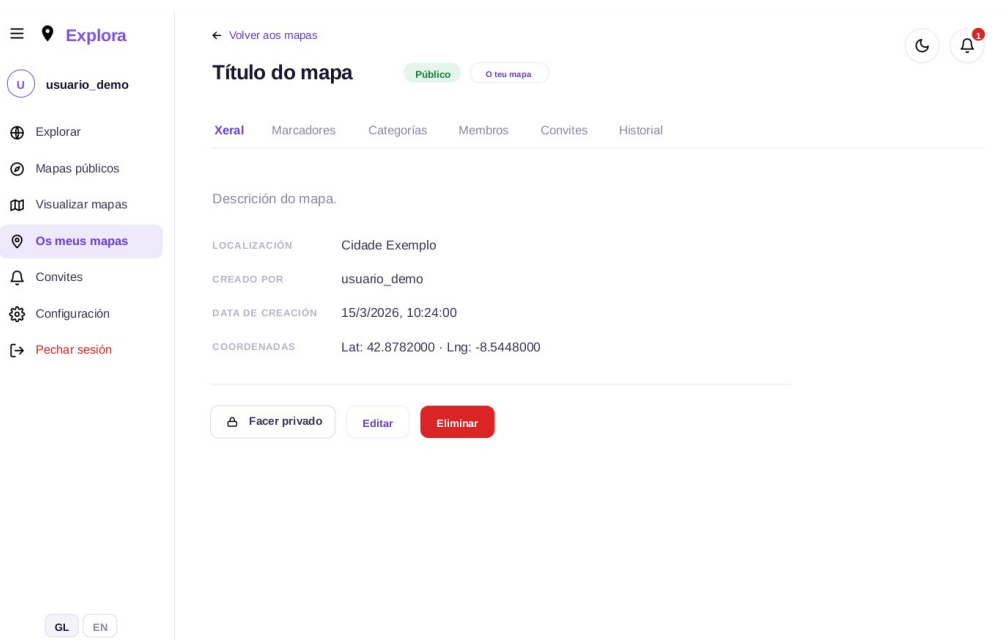
2. Diagrama da arquitectura do frontend



5. Mockups



Mapa: a vista principal como usuaria autenticada.



Detalle do mapa: vista de exemplo do detalle dun mapa propio.

Explora

usuario_demo

Explorar

Mapas públicos

Visualizar mapas

Os meus mapas

Convites

Configuración

Pegar sesión

GL EN

Os meus mapas

Novo mapa



Os meus mapas

Colaboracións

Gardados

Público

Propietaria



Título do mapa

Descrición do mapa.

Cidade Exemplo

Os meus mapas: vista de exemplo dos meus mapas, creados pola usuaria.

6. Guía de estilo

Identidade visual e cor

Filosofía e concepto

O sistema de cores de Explora Map está construído sobre tres premisas fundamentais derivadas do propósito da aplicación: servir de andamio visual para mapas colaborativos baseados en OpenStreetMap sen interferir co contido xeográfico, transmitir confianza e concentración para sesións longas de traballo, e respetar a privacidade evitando dependencias de servizos externos de fontes ou analítica.

A paleta xira arredor dun primario púrpura (258° HSL, saturación media S=68%) acompañado de neutros con ton violeta que forman todas as superficies.

Principio fundamental: A interface está ao servizo do mapa. Ningunha cor da interface debe competir coa información xeográfica. O morado úsase con moderación no armazón; o lenzo do mapa debe ser sempre o foco visual.

Paleta de cores

Cores principais: Púrpura



Token	HEX	Uso
--color-primary-50	#F5F2FD	Fondo hover sutil
--color-primary-100	#EDE8FB	Fondo estados activos claros
--color-primary-300	#A98CF0	Estados interactivos claros

Token	HEX	Uso
--color-primary-400	#9370EB	Hover de elementos primarios
--color-primary-500	#7C52E8	Botón primario, foco, nav activa
--color-primary-600	#6840D4	Hover do botón primario
--color-primary-700	#5530B8	Texto sobre fondos claros

Cores neutras: Gris violáceo



Token	HEX	Uso
--color-neutral-50	#F7F5FB	Fondo xeral da app (--color-bg)
--color-neutral-100	#EDEAF5	Fondo sidebar (--color-surface-sidebar)
--color-neutral-200	#DDD9EE	Bordos por defecto
--color-neutral-300	#C4BEE0	Bordos fortes, separadores
--color-neutral-400	#9E97C5	Texto placeholder
--color-neutral-500	#7A72A8	Texto secundario
--color-neutral-600	#5C5585	Texto terciario
--color-neutral-700	#453F60	Texto corpo (nivel AAA)

Token	HEX	Uso
--color-neutral-800	#2D2848	Texto secundario oscuro
--color-neutral-900	#18142E	Texto cabeceiras, case negro

Superficies e fondos

Token	HEX	Uso
--color-surface	#FFFFFF	Tarxetas, modais, fondos de formulario
--color-surface-sidebar	#F0EDF8	Fondo do panel lateral e barras de ferramentas
--color-surface-elevated	#F7F5FB	Superficies elevadas sobre o fondo
--color-bg	#F5F2FD	Fondo xeral da páxina

Cores semánticas

Éxito	Aviso	Erro	Información
			
#1E9E2C	#D98E00	#DC1B2F	#1D61DC

Token principal	HEX	Fondo / Bordo	Uso
--color-success	#1E9E2C	#EBF8EC / #A8DFA9	Confirmacións, accións completadas
--color-warning	#D98E00	#FFF8E6 / #F5D080	Avisos, accións irreversibles

Token principal	HEX	Fondo / Bordo	Uso
--color-error	#DC1B2F	#FDEBEC / #F5A8AE	Erros, validacións fallidas
--color-info	#1D61DC	#EAF0FD / #A3BEF5	Información neutra, estados pendent

Cores predefinidas para marcadores do mapa



Estas cores están reservadas exclusivamente para os marcadores no lenzo do mapa. Non deben usarse en botóns, badges nin elementos da interface.

Token	HEX	Nome
--marker-red	#C0392B	Rojo Pomegrenade
--marker-orange	#D4600A	Laranxa
--marker-yellow	#B8860B	Ámbar escuro
--marker-green	#2E7D32	Verde bosque
--marker-blue	#1565C0	Azul cobalto
--marker-purple	#6A1B9A	Púrpura escuro
--marker-pink	#AD1457	Carmesí

Token	HEX	Nome
--marker-slate	#37474F	Gris pizarra
--marker-teal	#00695C	Verde azulado escuro
--marker-navy	#283593	Azul mariño

Accesibilidade e contraste

Todo o texto debe cumprir como mínimo o nivel AA de WCAG 2.1. O texto do corpo aspira ao nivel AAA sempre que sexa posible.

Par de cores	Ratio	Nivel	Uso
#7C52E8 sobre #FFFFFF	4.6:1	AA	Botón primario, texto de enlace
#5530B8 sobre #FFFFFF	7.2:1	AAA	Texto corpo sobre branco
#453F60 sobre #FFFFFF	9.1:1	AAA	Texto principal en modo claro
#18142E sobre #FFFFFF	17:1	AAA	Cabeceiras en modo claro
#FFFFFF sobre #7C52E8	4.6:1	AA	Texto branco en botón primario
#9370EB sobre #FFFFFF	3.1:1	AA	Só para texto ≥18pt ou 14pt bold

Estratexia para daltonismo: a paleta evita depender de distincións vermello-verde puro. Cada par de cores diferénciase tanto en matiz como en luminosidade, garantindo polo menos dous canles de distinción. As cores semánticas sempre se acompañan dunha icona ou etiqueta de texto (nunca a cor como único diferenciador).

Reglas de combinación

Texto sobre fondos

Fondo	Cor de texto permitida	Cor de texto NON permitida
Branco #FFFFFF	neutral-700, neutral-900, primary-700	primary-300, primary-400
Sidebar #F0EDF8	neutral-700, neutral-900	neutral-400 (contraste insuficiente)
Primary-500 #7C52E8	Branco #FFFFFF	primary-300 (sen contraste)
Error-bg #FDEBEC	neutral-900, color-error (#DC1B2F)	primary-500

Principio de parejas semánticas: combinar sempre a cor semántica principal (500) co seu token de fondo (bg) para alertas e banners. Exemplo: .alert--error usa --color-error como borde/texto e --color-error-bg como fondo.

Estratexia de modo escuro

O modo escuro impleméntase mediante o atributo data-theme="dark" no elemento <html>, controlado por JavaScript desde useTemaStore. A preferencia gárdase en localStorage. Os fondos do modo escuro usan escuros profundos con ton violeta (#18142E como máis escuro) en lugar de negro puro, evitando o contraste agresivo e mantendo coherencia de marca.

Token	Modo claro	Modo escuro
--color-surface	#FFFFFF	#1F1A35
--color-surface-sidebar	#F0EDF8	#18142E
--color-bg	#F5F2FD	#18142E

Token	Modo claro	Modo escuro
--color-neutral-900	#18142E	#F7F5FB (invertido)
--color-primary-500	#7C52E8	#B39DF5 (aclorado)
--color-error	#DC1B2F	#F05060

CSS custom properties

Todas as cores do sistema están definidas en `src/assets/styles/global.css` dentro de `:root`. O modo escuro sobreescribe os tokens necesarios en `[data-theme="dark"]`. Extracto do ficheiro de tokens:

```
:root {  
  /* Primario */  
  --color-primary-500: #7C52E8;  
  --color-primary-600: #6840D4;  
  --color-primary-700: #5530B8;  
  /* Neutros */  
  --color-neutral-700: #453F60; /* texto corpo */  
  --color-neutral-900: #18142E; /* cabeceiras */  
  /* Superficies */  
  --color-surface-sidebar: #F0EDF8;  
  --color-bg: #F5F2FD;  
}  
[data-theme="dark"] {  
  --color-primary-500: #B39DF5;  
  --color-surface: #1F1A35;  
  --color-bg: #18142E;  
}
```

Tipografía e texto

Fontes seleccionadas

Inter: Fonte principal de interface

Deseñada por Rasmus Andersson. Sans-serif humanista optimizada para lexibilidade en pantallas de alta resolución. Licenza SIL Open Font License. Úsase en toda a interface: navegación, formularios, botóns, corpo de texto e cabeceiras.

Xustificación: a neutralidade expresiva de Inter evita que a tipografía compita co contido cartográfico. O seu deseño humanista aporta calidez suficiente para evitar o aspecto clínico doutras sans-serif xeométricas.

JetBrains Mono: Fonte monoespaciada

Deseñada por JetBrains. Optimizada para código e datos técnicos con ligaduras activadas. Licenza SIL Open Font License. Úsase exclusivamente para coordenadas xeográficas, fragmentos de código e datos técnicos.

Estratexia de carga

Ambas as fontes intégranse no bundle da aplicación mediante o paquete npm `@fontsource`. Non se realizan peticións a Google Fonts nin a ningún servizo externo, en coherencia co principio de privacidade por deseño. As fontes cárganse de forma síncrona antes do primeiro pintado, eliminando o FOUT (destello de fonte incorrecta).

```
import '@fontsource/inter/400.css';  
import '@fontsource/inter/500.css';  
import '@fontsource/inter/600.css';  
import '@fontsource/inter/700.css';  
import '@fontsource/jetbrains-mono/400.css';
```

Escala tipográfica

Token	Tamaño	Peso	Uso
display	2.25rem	700	Títulos de páxina principais
h1	1.875rem	700	Cabeceiras de seccións principais

Token	Tamaño	Peso	Uso
h2	1.5rem	700	Cabeceiras de seccións secundarias
h3	1.25rem	600	Títulos de paneis e tarxetas
h4	1.125rem	600	Subtítulos
body lg	1.125rem	400	Texto introdutorio
body	1rem	400	Texto principal
body sm	0.875rem	400	Texto secundario e etiquetas de formulario
caption	0.75rem	400	Pés de paneis e metadatos
label	0.875rem	500	Etiquetas de botón e navegación
overline	0.75rem	500	Categorías, seccións en maiúsculas
code	0.875rem	400	Código, coordenadas e datos técnicos (JetBrains Mono)

Reglas de uso e xerarquía

Construír a xerarquía visual con diferenzas de tamaño e peso, nunca con familias tipográficas distintas. Non se usa máis dunha familia por contexto (Inter para UI, JetBrains Mono estritamente para datos técnicos).

Correcto:

- Usar Inter para toda a interface: navegación, formularios, botóns e corpo de texto
- Usar JetBrains Mono exclusivamente para coordenadas xeográficas e código
- Manter interliñado leading-normal (1.5) para texto de corpo
- Usar pesos 600–700 só para cabeceiras e énfase importante

Incorrecto:

- Non usar pesos inferiores a 400 para texto de corpo
- Non subliñar texto que non sexa un hiperligazón
- Non usar cursiva para elementos de interface

CSS custom properties

```
--font-ui: 'Inter', system-ui, sans-serif;  
--font-code: 'JetBrains Mono', 'Fira Code', monospace;  
--text-xs: 0.75rem; --text-sm: 0.875rem;  
--text-base: 1rem; --text-lg: 1.125rem;  
--text-xl: 1.25rem; --text-2xl: 1.5rem;  
--text-3xl: 1.875rem; --text-4xl: 2.25rem;  
--font-regular: 400; --font-medium: 500;  
--font-semibold: 600; --font-bold: 700;  
--leading-tight: 1.25; --leading-normal: 1.5;
```

Iconografía e elementos gráficos

Sistema de iconas

Explora Map usa SVGs propios no estilo Feather Icons: trazo de 2px, fill="none", strokeLinecap e strokeLinejoin="round", viewBox 24×24. Todas as iconas permanentes están definidas como compoñentes React reutilizables en src/components/Iconas.jsx, garantindo consistencia visual e accesibilidade. Algunhas iconas de uso puntual (MoonIcon, SunIcon, HamburgerIcon) están definidas como SVG inline no compoñente que as usa.

Propiedades técnicas

Propiedade	Valor	Notas
Estilo	Outline (sen recheo)	fill="none" en todos os SVG
Grosor do trazo	2px	strokeWidth={2}

Propiedade	Valor	Notas
ViewBox	0 0 24 24	Estándar en todos
Tamaño por defecto	20×20px	Prop size (default 20) en Iconas.jsx
Tamaño en botóns pequenos	18×18px	
Cor	currentColor	Herda do elemento pai, adaptable a tema
Aliñación vertical	vertical-align: middle	Para aliñar con texto inline
Licenza	MIT (estilo Feather Icons)	

Catálogo de iconas

Iconas de navegación principal

Compoñente	Onde se usa	Acción / Significado
GlobelIcon	Sidebar + BottomNav	Páxina principal / exploración xeográfica
CompassIcon	Sidebar + BottomNav	Explorar mapas públicos
LayersIcon	Sidebar + BottomNav	Visualizar capas de mapas
PinIcon	Sidebar + BottomNav	Os meus mapas
BellIcon	Sidebar	Convites / notificacións
SettingsIcon	Sidebar	Configuración da aplicación
LogoutIcon	Sidebar	Pechar sesión

Compoñente	Onde se usa	Acción / Significado
LoginIcon	Sidebar + BottomNav	Iniciar sesión (sen autenticar)
UserIcon	BottomNav	Perfil de usuaria

Iconas de acción (definidas en Iconas.jsx)

Compoñente	Acción	Contexto de uso
TrashIcon	Eliminar	Marcadores, categorías, membros
EyelIcon	Ver / Previsualizar	Tarxetas en ExplorarMapasPage
BookmarkIcon	Gardar mapa	Tarxetas en ExplorarMapasPage (estado non gardado)
BookmarkFilledIcon	Mapa gardado	Tarxetas en ExplorarMapasPage (estado gardado)
SettingsIcon	Editar / Config	Botón de edición de mapa en detalle

Iconas inline (SVG definido directamente no compoñente)

Icona	Compoñente	Uso
Hamburger (tres liñas)	≡ Sidebar.jsx	Colapsar/expandir sidebar (mesmo icono en ambos estados)
MoonIcon	AppLayout.jsx	Activar modo escuro
SunIcon	AppLayout.jsx	Activar modo claro
CheckIcon	Varios	Confirmación, estado correcto

Icona	Compoñente	Uso
XIcon / ClosesIcon	Modais, paneis	Pechar / Cancelar

Regras de uso

- Usar SettingsIcon tanto para Configuración na nav como para editar mapas. É a icona de "xestión" xenérica
- TrashIcon exclusivamente para eliminar, nunca para outras accións destrutivas
- Non usar BookmarkIcon para nada que non sexa gardar/desgardar mapas
- As cores dos marcadores de mapa NON se usan nos iconos de interface
- Todos os iconos herdan a cor do elemento pai mediante `currentColor`, non se codifica cor directamente

Layout, espaciado e responsive

Sistema de espaciado (base 4px)

Token CSS	Valor	Uso típico
--space-1	0.25rem (4px)	Separación mínima entre elementos inline
--space-2	0.5rem (8px)	Gap entre icona e etiqueta, padding de badge
--space-3	0.75rem (12px)	Padding interno de botóns pequenos
--space-4	1rem (16px)	Padding estándar de tarxetas e seccións
--space-5	1.25rem (20px)	Padding intermedio
--space-6	1.5rem (24px)	Separación entre grupos de elementos

Token CSS	Valor	Uso típico
--space-8	2rem (32px)	Marxe entre seccións
--space-10	2.5rem (40px)	Separación maior
--space-12	3rem (48px)	Separación maior entre bloques de contido
--space-16	4rem (64px)	Separacións grandes (cabeceiras de páxina)
--space-20	5rem (80px)	Separacións moi grandes

Tokens de layout

Token	Valor	Uso
--sidebar-collapsed-width	56px	Sidebar contraído (só iconas)
--sidebar-expanded-width	240px	Sidebar expandido (iconas + etiquetas)
--topbar-height	52px	Altura da MobileHeader (só móbil ≤640px)
--bottom-nav-height	64px	Altura da navegación inferior (só móbil ≤640px)

Breakpoints responsive

A estratexia é Desktop First. O layout completo con sidebar lateral é o estado base; os breakpoints reducen e reorganizan para dispositivos máis pequenos.

Breakpoint	Valor	Comportamento
Móbil	≤ 640px	Sidebar oculto. MobileHeader (52px) na parte superior + BottomNav (64px) na parte inferior. O contido ocupa o 100% da pantalla.

Breakpoint	Valor	Comportamento
Tablet	641px – 900px	Sidebar visible pero colapsado por defecto (56px de ancho). Sen BottomNav.
Desktop	> 900px	Sidebar expandido por defecto (240px). Sen MobileHeader nin BottomNav.

Sistema de compoñentes básicos

Esta sección mostra como se aplican os tokens de deseño en código CSS real. Todos os compoñentes están definidos en `src/assets/styles/global.css` e `src/assets/styles/mapas.css`.

Botóns

Variantes e estados

Clase	Estado normal	Hover	Disabled
.btn--primary	bg: primary-500, text: white	bg: primary-600	opacity: 0.5, cursor: not-allowed
.btn--secondary	bg: primary-100, border: primary-500, text: primary-700	bg: primary-200	opacity: 0.5
.btn--ghost	bg: transparent, text: neutral-700	bg: neutral-100	opacity: 0.5
.btn--danger	bg: error, text: white	bg: error escurecido	opacity: 0.5
.btn--warning	bg: warning, text: white	bg: warning escurecido	opacity: 0.5

Tamaños

Clase modificadora	Tamaño	Uso
.btn--sm	padding: 6px 12px, font: 0.8125rem	Accións secundarias en tarxetas e paneis
.btn (base)	padding: 10px 20px, font: 0.9375rem	Accións estándar
.btn--lg	padding: 14px 28px, font: 1rem	Accións principais en páxinas (submit de formulario)
.btn--icon	padding: 8px, ratio: 1:1	Botóns con só icona (pechar modal)

Estado de foco (accesibilidade): todos os botóns teñen outline: none ao recibir foco e en cambio mostran box-shadow: --shadow-focus (0 0 0 3px rgba(124,82,232,0.25)), cumprindo WCAG 2.4.11 con 3px de separación visible.

Formularios e inputs

Estrutura dun campo de formulario

```
<div class="form-group">  
  <label class="form-label form-label--required">Nome</label>  
  <input class="form-input form-input--error" />  
  <span class="form-error">Campo obrigatorio</span>  
  <span class="form-hint">Máximo 50 caracteres</span>  
</div>
```

Estados do input

Estado	Estilos aplicados
Normal	border: --border-default (neutral-200), bg: white, radius: --radius-md
Hover	border: --border-strong (neutral-300)

Estado	Estilos aplicados
Focus	border: --border-focus (primary-500), box-shadow: --shadow-focus, outline: none
Error	border: --color-error, bg: --color-error-bg
Error focus	border: --color-error (mantén cor de erro), box-shadow: rgba(error,0.25)
Disabled	bg: neutral-50, cursor: not-allowed, opacity: 0.7

Etiqueta obligatoria: `.form-label--required::after` engade o asterisco (*) en `--color-error` de forma automática mediante CSS, sen necesidade de marcado HTML adicional.

Tarxetas e contedores

MapaCard (.mapa-card)

Tarxeta principal da aplicación, usada en `MapaListPage` e `ExplorarMapasPage`.
Propiedades base:

- background: --color-surface (branco)
- border: --border-default (1px neutral-200)
- border-radius: --radius-lg (0.75rem)
- box-shadow: --shadow-sm
- hover: box-shadow: --shadow-md, translateY(-1px), transition: var(--duration-normal)

Badges (.badge)

Usados para indicar estado de convites, visibilidade de mapa e rol de membro. Todos teñen `border-radius: --radius-full` e `padding: 2px 8px`.

Clase	Cores	Uso
.badge--publico	bg: primary-50, text: primary-700, border: primary-200	Mapa público
.badge--privado	bg: neutral-100, text: neutral-700, border: neutral-300	Mapa privado
.badge--pendente	bg: warning-bg, text: warning, border: warning-border	Convite pendente
.badge--aceptado	bg: success-bg, text: success, border: success-border	Convite aceptado
.badge-- rexeitado / .badge --cancelado	bg: error-bg, text: error, border: error- border	Convite rexeitado ou cancelado
.badge--expirado	bg: neutral-100, text: neutral-500, border: neutral-200	Convite expirado

Arquitectura CSS e código

Principios de código

O sistema de estilos de Explora Map está organizado en ficheiros CSS planos modulares importados globalmente en src/main.jsx. Non se usa ningún framework CSS (Tailwind, Bootstrap). Todo o sistema está construído sobre CSS custom properties (tokens de deseño) definidas en global.css.

Convencións de nomenclatura

BEM adaptado con patrón bloque__elemento--modificador.

```
.mapa-card {} /* bloque */
.mapa-card__header {} /* elemento */
.mapa-card--destacado {} /* modificador */
```

Principios aplicados

- Tokens primeiro: ningunha cor, tamaño ou sombra está hardcoded — todo usa variables CSS de :root
- Modo escuro mediante [data-theme="dark"] no <html>, sobreescribindo tokens de cor en global.css
- Non se usa !important salvo para sobreescribir estilos inline de Leaflet (casos documentados)
- Non hai CSS Modules: as clases son globais e o alcance xestiónase mediante nomenclatura BEM

Estrutura de ficheiros

Ficheiro	Contido
global.css	Tokens (cor, tipo, espaciado, sombras, z-index, movement, layout). Sistema de botóns (.btn), formularios (.form-*), badges (.badge), utilidades (.sr-only, .truncate). Reset e base.
sidebar.css	Panel lateral, MobileHeader, BottomNav. Inclúe selector de idioma e tema no sidebar.
mapas.css	Páxinas de lista e detalle de mapas (.mapa-card, .detalle_*), modais (.form-modal), selector de cor (.cor-selector), sistema de marcadores, tabs, paneis de convite/membro/categoría.
mapa-principal.css	Páxina principal de mapa a pantalla completa (.panel-cidade, .marcador-popup-flotante).
auth.css	Páxinas de login, rexistro e verificación (.auth-card, .field_*).
perfil.css	Páxina de perfil de usuaria (.perfil-*, .perfil-avatar).
configuracion.css	Páxina de configuración (.config-*, .idioma-option, .tema-option).

Ficheiro	Contido
explorar.css	Páxina de exploración de mapas públicos (.explorar-card).
historial.css	Panel de historial de cambios (.historial-*, badges de acción).
notificacions.css	Panel de notificacións (.notificacions-*, .notificacion-item).
confirm-dialog.css	Diálogo de confirmación global (.confirm-overlay, .confirm-dialog).
visualizar-panel.css	Panel drawer de visualización de mapas (.visualizar-panel, .mapa-toggle-item).
map-search.css	Barra de busca de localizacións (.map-search__*).
verificar.css	Páxina de verificación de conta (.verificar-card).

Incidencias de código detectadas e plan de refactorización

As seguintes incidencias foron identificadas mediante auditoría do código CSS.

Incidencia	Descrición e solución proposta
.btn duplicado	A clase .btn e as variantes .btn--primary, .btn--secondary, .btn--ghost, .btn--danger están definidas en global.css e redefinidas en mapas.css con estilos distintos. Isto crea conflitos de especificidade. Solución: eliminar todas as definicións de .btn en mapas.css e usar exclusivamente as de global.css.
--color-text-secondary non definida	Usada con fallback #888 en ExplorarMapasPage.jsx e MapaPrincipalPage.jsx. Non está definida en :root. Solución: engadir --color-text-secondary: var(--color-neutral-500)

Incidencia	Descrición e solución proposta
	en :root de global.css.
.badge duplicado	Definido en global.css e en mapas.css. Mesmo problema que .btn. Solución: eliminar de mapas.css.
.form-group/.form-label/.form-input duplicados	Definidos en global.css e redefinidos en perfil.css con diferenzas. Solución: eliminar de perfil.css e axustar perfil-specific overrides con selectores máis específicos (.perfil-form .form-input).
.mapa-toggle-item__btn-engadir dobre definición	Definido dúas veces en visualizar-panel.css con estilos contraditorios. A segunda definición sobreescribe a primeira. Solución: unificar nunha soa regra.
.detalle__tab-accions--marcadores dobre definición	Definido dúas veces en mapas.css (flex-end e flex-start). Solución: revisar o comportamento esperado e deixar só a definición correcta.
.map-search__dropdown sen modo claro	Referenciadas en [data-theme="dark"] pero sen definición en modo claro. Posible inconsistencia visual. Solución: engadir a definición base.
Carga de fontes non documentada	Inter e JetBrains Mono non están cargadas en ningún ficheiro CSS auditado. A carga debe facerse en index.html ou mediante @fontsource en main.jsx. Requírese verificación.